การปรับปรุงประสิทธิภาพของระบบสารสนเทศโดยการเพิ่ม กระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ

Improving Information System Efficiency by Implementing InputValidation and Automated Testing

ปิติภัทร กิจสวน

วิทยานิพนธ์ระดับบัณฑิตปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิทยาศาสตร์บัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
ภาคเรียนที่ 1 ปีการศึกษา 2568
ลิขสิทธิ์เป็นของคณะวิทยาศาสตร์ มหาวิทยาลัยนเรศวร

วิทยานิพนธ์ระดับปริญญาตรี

เรื่อง การปรับปรุงประสิทธิภาพของระบบสารสนเทศโดยการเพิ่ม
กระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ
ของ นายปิติภัทร กิจสวน
ได้รับการพิจารณาให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิทยาศาสตร์บัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

คณะกรรมการสอบวิทยานิพนธ์ระดับปริญญาตรี



.....อาจารย์ที่ปรึกษา

(ผู้ช่วยศาสตราจารย์ ดร.วินัย วงษ์ไทย)



.....กรรมการ

(ผู้ช่วยศาสตราจารย์ ดร.วันสุรีย์ มาศกรัม)



.....กรรมการ

(อาจารย์ อดิเรก รุ่งรังษี)

.....

(อาจารย์ วุฒิพงษ์ เรือนทอง)

หัวหน้าภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ

วันที่ 10 เดือน พฤศจิกายน ปี พ.ศ. 2568

ชื่อเรื่อง การปรับปรุงประสิทธิภาพของระบบสารสนเทศโดยการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ
ผู้วิจัย นายปิติภัทร กิจสวน รหัสนิสิต 65313037
อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์ ดร.วินัย วงษ์ไทย
ประเภทสารนิพนธ์ วิทยานิพนธ์ระดับปริญญาตรี วท.บ.วิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยนเรศวร
คำสำคัญ ระบบสารสนเทศ การตรวจสอบข้อมูลนำเข้า
การทดสอบแบบอัตโนมัติ SonarQube

บทคัดย่อ

งานวิจัยนี้มีจุดมุ่งหมายเพื่อปรับปรุงประสิทธิภาพและคุณภาพของระบบสารสนเทศ โดยการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า และการทดสอบแบบอัตโนมัติ เพื่อให้ระบบมีความถูกต้อง เสถียร และลดข้อผิดพลาดจากการทำงานของผู้ใช้และโปรแกรม โดยระบบต้นแบบที่ใช้ในการทดลองคือระบบลงทะเบียนเรียนที่พัฒนาด้วย Django Framework ซึ่งเป็นโครงสร้างแบบ Model–View–Template ที่ช่วยแบ่งส่วนการทำงานอย่างเป็นระบบ การวิจัยนี้แบ่งออกเป็นสองส่วนหลัก ได้แก่ (1) การเพิ่มกลไกการตรวจสอบข้อมูลนำเข้าในส่วนของแบบฟอร์ม เพื่อควบคุมรูปแบบและความถูกต้องของข้อมูลก่อนจัดเก็บ และ (2) การเพิ่มกระบวนการทดสอบอัตโนมัติ โดยใช้เครื่องมือ Pytest สำหรับการทดสอบระดับหน่วย และการทดสอบแบบบูรณาการ ร่วมกับ Robot Framework สำหรับการทดสอบแบบ End-to-End เพื่อจำลองการทำงานของผู้ใช้จริงในระบบ จากนั้นได้ใช้เครื่องมือ SonarQube ในการวิเคราะห์คุณภาพของซอร์สโค้ดเชิงสถิติ เพื่อประเมินตัวชี้วัดหลัก ได้แก่ 1.ความน่าเชื่อถือ (Reliability) 2.ความสามารถในการบำรุงรักษา (Maintainability) 3. ความครอบคลุมของการทดสอบ (Coverage) 4.ระดับความซ้ำซ้อนของโค้ด (Duplications) โดยเปรียบเทียบระหว่างระบบก่อนการปรับปรุง และระบบหลังการปรับปรุง ผลการทดลองพบว่า ระบบหลังการปรับปรุง มีค่า Reliability ลดลงจาก 3 เหลือ 0, ค่า Maintainability ลดลงจาก 37 เหลือ 33, ค่า Coverage เพิ่มขึ้นจาก 0% เป็น 78.3%, และค่า Duplications ลดลงจาก 8.0% เหลือเพียง 7.2% ซึ่งแสดงให้เห็นว่าการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบอัตโนมัติช่วยยกระดับคุณภาพซอร์สโค้ด ทั้งในด้านความน่าเชื่อถือ ความสามารถในการบำรุงรักษา และลดความผิดพลาดในการพัฒนาโปรแกรม

Title Improving Information System Efficiency by Implementing Input Validation and Automated Testing

Author Mr. Pithiphat Khitsuan Student ID. 65313037

Advisor Assistant Professor Dr. Winai Wongthai

Academic Paper Undergraduate Thesis B.S. (Computer Science)
Faculty of Science, Naresuan University

Keywords Information System Input Validation Automated Testing Django SonarQube

Abstract

This research aims to improve the efficiency and quality of an information system by implementing input validation and automated testing processes to enhance system accuracy and stability. A prototype course registration system was developed using the Django Framework based on the Model–View–Template (MVT) architecture. Input validation was applied through Django's clean() function and validators to ensure data correctness, while automated testing was implemented using Pytest for unit and integration tests, and Robot Framework for end-to-end testing to simulate real user operations. The system's source code quality was evaluated using SonarQube, measuring key metrics including Bugs, Code Smells, and Test Coverage, comparing results between the pre-improvement (T0) and post-improvement (T1) versions. The results show that after the improvement, the system achieved significant quality enhancement: the Reliability score decreased from 3 to 0, the Maintainability score decreased from 37 to 33, the Test Coverage increased from 0% to 78.3%, and the Code Duplications decreased from 8.0% to 7.2%.
These results indicate that the integration of input validation and automated testing effectively enhances source code quality improving reliability, maintainability, and reducing potential development errors.

กิตติกรรมประกาศ

การศึกษาวิจัยครั้งนี้สำเร็จลุล่วงได้ด้วยความกรุณาและการสนับสนุนจากหลายฝ่าย ผู้วิจัยขอกราบขอบพระคุณ อาจารย์ที่ปรึกษาที่ได้กรุณาให้คำแนะนำ ตรวจสอบ และปรับปรุงแนวทางการดำเนินงานวิจัยอย่างใกล้ชิดตลอดระยะเวลา ทั้งยังได้ให้คำปรึกษา คำชี้แนะ และแนวทางในการแก้ไขปัญหาอันเป็นประโยชน์อย่างยิ่ง ทำให้งานวิจัยฉบับนี้สำเร็จลุล่วงไปด้วยดี

ผู้วิจัยขอขอบพระคุณ คณาจารย์ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยนเรศวรที่ได้มอบความรู้ ประสบการณ์ และแรงบันดาลใจในการเรียนรู้ตลอดระยะเวลาที่ศึกษาเล่าเรียน รวมทั้งขอขอบคุณ เพื่อนนิสิตและผู้ร่วมโครงงาน ที่ให้ความช่วยเหลือและแลกเปลี่ยนความคิดเห็นอย่างจริงใจตลอดการดำเนินงาน

สุดท้ายนี้ ผู้จัดทำขอขอบพระคุณ ครอบครัว ที่เป็นกำลังใจสำคัญ คอยให้การสนับสนุนและให้คำปรึกษาในทุกช่วงเวลาตลอดจนผู้ที่มีส่วนเกี่ยวข้องทุกท่านที่ช่วยให้งานวิจัยฉบับนี้สำเร็จลุล่วงไปด้วยดี ขอกราบขอบพระคุณอย่างสูงมา ณ โอกาสนี้

ปิติภัทร กิจสวน

สารบัญ

	หน้า
บทคัดย่อ ภาษาไทย	ก
บทคัดย่อ (ภาษาอังกฤษ)	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	ฉ
สารบัญรูปภาพ	ช
บทที่ 1 บทนำ	1
1.1 ความเป็นมาของปัญหา	2
1.2 วัตถุประสงค์ของการวิจัย	3
1.3 ขอบเขตของงานวิจัย	3
1.4 สมมุติฐานงานวิจัย	4
1.5 กรอบแนวความคิดในการวิจัย	4
1.6 คำสำคัญของการวิจัย	5
1.7 ประโยชน์ที่คาดว่าจะได้รับ	6
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	7
2.1 ทฤษฎีที่เกี่ยวข้อง	7
2.2 งานวิจัยที่เกี่ยวข้อง	17
บทที่ 3 วิธีดำเนินงานวิจัย	19
3.1 การศึกษาทฤษฎีที่เกี่ยวข้องกับงานวิจัย หรือ การศึกษาและวิเคราะห์ปัญหางานวิจัย	19
3.2 การรวบรวมและเตรียมข้อมูล	19
3.3 เครื่องมือที่ใช้ในการทำวิจัยและขั้นตอนการทำวิจัย	20
3.4 การออกแบบและพัฒนาระบบต้นแบบ	21

สารบัญ (ต่อ)

	หน้า
3.5 วิธีการทดสอบและวิเคราะห์ผล	22
บทที่ 4 ผลการวิจัยและการอภิปรายผล	25
4.1 ผลการพัฒนาระบบต้นแบบ	25
4.2 ผลการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation)	27
4.3 ผลการทดสอบแบบอัตโนมัติ (Automated Testing)	28
4.4 ผลการประเมินคุณภาพของซอร์สโค้ดด้วย SonarQube	29
4.5 การวิเคราะห์ผลการปรับปรุงระบบ	34
บทที่ 5 สรุปผลการวิจัยและข้อเสนอแนะ	36
5.1 สรุปผลการวิจัย	36
5.2 ข้อจำกัดของงานวิจัย	37
5.3 ข้อเสนอแนะงานวิจัย	38
บรรณานุกรม	40
ภาคผนวก	42
ประวัติผู้วิจัย	46

สารบัญตาราง

ตาราง		หน้า
1.1	กรอบแนวความคิดของงานวิจัย	4
4.1	ผลการประเมินคุณภาพของซอร์สโค้ดก่อนและหลังการปรับปรุงระบบด้วยเครื่องมือ SonarQube	31

สารบัญรูป

รูป		หน้า
2.1	Testing Pyramid	9
4.1	สถาปัตยกรรมของระบบ Django Framework ที่เพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Tests)	26
4.2	ตัวอย่างผลการตรวจสอบข้อมูลนำเข้าในระบบต้นแบบ	27
4.3	ผลการทดสอบด้วย Pytest	28
4.5	ผลการประเมินคุณภาพของซอร์สโค้ดก่อนและหลังการปรับปรุงระบบด้วย SonarQube	33

บทที่ 1
บทนำ

ในปัจจุบัน ระบบสารสนเทศมีบทบาทสำคัญต่อการดำเนินงานในองค์กรต่าง ๆ โดยเฉพาะในสถาบันการศึกษา หน่วยงานราชการ และภาคธุรกิจ อย่างไรก็ตาม ปัญหาที่มักพบได้ทั่วไปคือระบบสารสนเทศจำนวนมากขาดกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) ที่รัดกุม และไม่มีการทดสอบซอฟต์แวร์ (Software Testing) อย่างเป็นระบบ ส่งผลให้เกิดข้อผิดพลาดในการทำงานของระบบ เช่น การบันทึกข้อมูลไม่ถูกต้อง การเกิดบัคในขั้นตอนการประมวลผล และความยากในการบำรุงรักษาในระยะยาว

เพื่อแก้ไขปัญหาดังกล่าว ผู้วิจัยจึงได้ศึกษาแนวทางการเพิ่มประสิทธิภาพของระบบสารสนเทศ โดยประยุกต์ใช้กระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบพลวัตอัตโนมัติ (Automated Dynamic Testing) ภายใต้กรอบแนวคิดของการพัฒนาซอฟต์แวร์สมัยใหม่ที่เน้นการประกันคุณภาพตั้งแต่ขั้นตอนการ ทั้งนี้เพื่อให้สามารถตรวจสอบคุณภาพของระบบได้อย่างต่อเนื่องและลดความผิดพลาดจากการพัฒนาโดยมนุษย์

ในการดำเนินการวิจัย ผู้วิจัยได้สร้างระบบต้นแบบ (Prototype System) ขึ้นเพื่อจำลองกระบวนการทำงานของระบบสารสนเทศทั่วไป โดยเลือกใช้ระบบลงทะเบียนเรียนที่พัฒนาด้วย Django Framework เป็นกรณีศึกษา เนื่องจากเป็นระบบที่มีโครงสร้างการทำงานครบถ้วนและมีความสัมพันธ์ของข้อมูลหลายระดับ จึงเหมาะสมสำหรับการทดสอบแนวทางที่เสนอ โดยในระบบต้นแบบนี้ได้เพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) เพื่อควบคุมความถูกต้องของข้อมูลก่อนบันทึก และเพิ่มกระบวนการทดสอบแบบอัตโนมัติในหลายระดับ ได้แก่ Unit Test Integration Test และ End-to-End Test (E2E Test) ด้วยเครื่องมือ Pytest และ Robot Framework

จากนั้นทำการประเมินผลโดยใช้ SonarQube ซึ่งเป็นเครื่องมือสำหรับวิเคราะห์คุณภาพซอร์สโค้ดในมิติของความน่าเชื่อถือของระบบ (Reliability) ความสามารถในการบำรุงรักษา (Maintainability) ความครอบคลุมของการทดสอบ (Coverage) และอัตราการซ้ำซ้อนของโค้ด (Duplications) ผลลัพธ์จากการประเมินจะถูกนำมาเปรียบเทียบระหว่างระบบก่อนและหลังการปรับปรุง (T0 และ T1) เพื่อวัดประสิทธิภาพของแนวทางที่นำเสนอว่าช่วยเพิ่มคุณภาพของระบบสารสนเทศได้มากเพียงใด

1.1 ความเป็นมาของปัญหา

ในปัจจุบัน ระบบสารสนเทศมีบทบาทสำคัญต่อการดำเนินงานในองค์กรต่าง ๆ ระบบลงทะเบียนเรียน ซึ่งเกี่ยวข้องโดยตรงกับการจัดการข้อมูลของผู้ใช้และรายวิชา ความถูกต้องและความเสถียรของระบบจึงเป็นสิ่งจำเป็นต่อการดำเนินงานอย่างมีประสิทธิภาพ การทดสอบซอฟต์แวร์ (Software Testing) เป็นกระบวนการสำคัญที่ช่วยประเมินคุณภาพของระบบก่อนนำไปใช้งานจริง เพื่อให้มั่นใจว่าระบบสามารถทำงานได้ตรงตามที่ออกแบบไว้ (I. Foundation, 2023)

การทดสอบซอฟต์แวร์แบ่งออกเป็นสองลักษณะหลัก ได้แก่ การทดสอบแบบสถิต (Static Testing) และการทดสอบแบบพลวัต (Dynamic Testing) โดยการทดสอบแบบสถิต เป็นการตรวจสอบโค้ดหรือเอกสารโดยไม่ต้องรันโปรแกรม เช่น การตรวจสอบโครงสร้างของซอร์สโค้ดหรือการใช้เครื่องมือวิเคราะห์โค้ด (Static Analysis) ขณะที่การทดสอบแบบพลวัต เป็นการรันระบบจริงเพื่อสังเกตผลลัพธ์และพฤติกรรมของระบบภายใต้เงื่อนไขต่าง ๆ (I. Foundation, 2023)

อย่างไรก็ตาม การทดสอบแบบแมนนวล (Manual Testing) มักใช้เวลาและแรงงานมาก อีกทั้งยังมีโอกาสเกิดข้อผิดพลาดจากมนุษย์ ซึ่งไม่เหมาะกับระบบที่ต้องมีการปรับปรุงและทดสอบซ้ำบ่อยครั้ง (Thoughtworks, 2012) เพื่อเพิ่มประสิทธิภาพและลดความผิดพลาด จึงมีการนำแนวคิดการทดสอบอัตโนมัติ (Automated Testing) เข้ามาใช้ โดยเฉพาะในส่วนของการทดสอบแบบพลวัต ซึ่งสามารถรันโปรแกรมจริงและประเมินผลการทำงานของระบบได้อย่างถูกต้องและรวดเร็ว

ในงานวิจัยนี้ ผู้วิจัยได้สร้าง ระบบต้นแบบ (Prototype System) ขึ้นโดยใช้ Django Framework เพื่อใช้เป็นกรณีศึกษาสำหรับการทดลองแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ โดย Django เป็น Framework ที่รองรับสถาปัตยกรรมแบบ Model–View–Template (MVT) ช่วยแยกส่วนการทำงานของระบบอย่างชัดเจนและเอื้อต่อการทดสอบ (Django Software, 2024) ระบบต้นแบบนี้ได้ถูกพัฒนาให้รองรับกระบวนการทดสอบอัตโนมัติด้วย Pytest สำหรับการทดสอบระดับหน่วย (Unit Test) และการทดสอบแบบบูรณาการ (Integration Test) (Krekel, 2024) รวมถึง Robot Framework (Selenium) สำหรับการทดสอบแบบ End-to-End (E2E) ซึ่งเป็นการจำลองพฤติกรรมการใช้งานของผู้ใช้จริง (Robot Framework, 2024)

นอกจากการทดสอบแบบพลวัตแล้ว งานวิจัยยังให้ความสำคัญกับกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) เพื่อควบคุมความถูกต้องของข้อมูลก่อนจัดเก็บเข้าสู่ระบบ ซึ่งเป็นแนวทางที่สอดคล้องกับหลักการของ OWASP Foundation (2023) ที่ระบุว่าการตรวจสอบข้อมูลนำเข้าช่วยลดข้อผิดพลาดของผู้ใช้และลดความเสี่ยงจากการโจมตีทางไซเบอร์ Django Framework มีฟังก์ชันในตัว เช่น clean() และ validators ที่ช่วยให้สามารถกำหนดรูปแบบข้อมูลได้อย่างปลอดภัยและมีประสิทธิภาพ (Django Software, 2024)

เพื่อประเมินคุณภาพของระบบทั้งก่อนและหลังการปรับปรุง ผู้วิจัยใช้เครื่องมือ SonarQube ในการวิเคราะห์แบบ Static Testing เพื่อวัดค่าตัวชี้วัดหลัก ในเวอร์ชันปัจจุบัน SonarQube ได้พัฒนาการประเมินคุณภาพซอร์สโค้ดให้เป็นไปตามมาตรฐาน ISO/IEC 25010 โดยแปลงค่าตัวชี้วัด Bugs และ Code Smells ให้แสดงผลในรูปแบบ Reliability และ Maintainability เพื่อสะท้อนคุณภาพในเชิงคุณลักษณะของซอฟต์แวร์ได้อย่างชัดเจนยิ่งขึ้น (SonarSource, 2024) ผลการประเมินดังกล่าวจะถูกนำมาใช้เพื่อเปรียบเทียบคุณภาพระหว่างระบบก่อนปรับปรุง (T0) และระบบหลังปรับปรุง (T1) เพื่อพิสูจน์ว่าแนวทางที่นำเสนอสามารถเพิ่มประสิทธิภาพของระบบสารสนเทศได้จริง

จากแนวคิดดังกล่าว งานวิจัยนี้จึงมุ่งเน้นการใช้ระบบต้นแบบเพื่อทดลองการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบพลวัตอัตโนมัติ โดยมีเป้าหมายเพื่อปรับปรุงคุณภาพของระบบสารสนเทศในมิติของความถูกต้อง ความเสถียร และความสามารถในการบำรุงรักษาในระยะยาว

1.2 วัตถุประสงค์ของการวิจัย

1. เพื่อปรับปรุงประสิทธิภาพของระบบสารสนเทศ

1.3 ขอบเขตของงานวิจัย

1. โครงงานนี้เป็นการพัฒนา ระบบลงทะเบียนเรียนต้นแบบ ด้วย Django Framework เพื่อใช้เป็นระบบจำลองในการทดสอบและประเมินคุณภาพของซอฟต์แวร์เท่านั้น

2. ระบบได้รับการเพิ่มกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) โดยใช้ฟังก์ชันและคลาสใน Django เพื่อควบคุมรูปแบบและความถูกต้องของข้อมูลที่ผู้ใช้กรอกเข้าสู่ระบบ

3. ระบบได้รับการเพิ่มกระบวนการ การทดสอบแบบพลวัตอัตโนมัติ (Automated Dynamic Testing) เพื่อประเมินการทำงานของระบบจริง โดยใช้ Pytest สำหรับการทดสอบระดับหน่วย (Unit Test) และการทดสอบแบบบูรณาการ (Integration Test) และ Robot Framework (Selenium) สำหรับการทดสอบแบบ End-to-End (E2E) ซึ่งจำลองพฤติกรรมการใช้งานของผู้ใช้

4. การประเมินคุณภาพของซอร์สโค้ดใช้เครื่องมือ SonarQube โดยพิจารณาตัวชี้วัดหลักที่เกี่ยวข้องกับงานวิจัย ได้แก่ Reliability, Maintainability, Coverage และ Duplications เพื่อวัดคุณภาพของโค้ดก่อนและหลังการปรับปรุงระบบ

5. การทดสอบและประเมินผลทั้งหมดดำเนินการภายใน สภาพแวดล้อมจำลอง (Local Environment) เพื่อควบคุมปัจจัยที่อาจมีผลต่อผลลัพธ์การทดลอง

6. โครงงานนี้ ไม่ครอบคลุม การทดสอบด้านประสิทธิภาพของระบบ ความปลอดภัยของระบบ หรือการเชื่อมต่อกับระบบจริง เนื่องจากขอบเขตของการศึกษาเน้นเฉพาะการปรับปรุงประสิทธิภาพผ่านกระบวนการทดสอบแบบพลวัตและการตรวจสอบข้อมูลนำเข้า

1.4 สมมุติฐานงานวิจัย

การเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบพลวัตอัตโนมัติในระบบต้นแบบ จะช่วยปรับปรุงประสิทธิภาพของระบบสารสนเทศในด้านคุณภาพของซอร์สโค้ดและความถูกต้องของการทำงาน เมื่อประเมินผลด้วย SonarQube จะพบว่ามีการลดลงของจำนวน Reliability, Maintainability และ Duplications และมีค่า Coverage ที่สูงขึ้นเมื่อเทียบกับระบบเดิม

1.5 กรอบแนวความคิดในการวิจัย

การวิจัยครั้งนี้มุ่งเน้นการพัฒนาและประเมินแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ โดยใช้ ระบบต้นแบบ (Prototype System) ที่พัฒนาด้วย Django Framework เป็นกรณีศึกษา เพื่อทดสอบผลของการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Dynamic Testing) ต่อประสิทธิภาพของระบบ

กระบวนการดำเนินการเริ่มจากการสร้างระบบต้นแบบในสภาพแวดล้อมเดียวกันสองชุด ได้แก่ ระบบก่อนปรับปรุง (T0) ไม่มีการตรวจสอบข้อมูลนำเข้าและไม่มีการทดสอบอัตโนมัติ และระบบหลังปรับปรุง (T1) เพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบอัตโนมัติ

ระบบทั้งสองจะถูกประเมินด้วยเครื่องมือ SonarQube เพื่อวิเคราะห์คุณภาพของซอร์สโค้ดในเชิงสถิติ โดยใช้ตัวชี้วัดหลัก ได้แก่ Reliability, Maintainability, Coverage และ Duplications ผลลัพธ์ที่ได้จะนำมาเปรียบเทียบเพื่อวัดอัตราการปรับปรุง (Percentage of Improvement) ของคุณภาพระบบ ดังแสดงในตารางที่ 1.1

ตาราง 1.1 กรอบแนวความคิดของงานวิจัย

องค์ประกอบ	ระบบก่อนปรับปรุง (T0)	ระบบหลังปรับปรุง (T1)	เครื่องมือที่ใช้
การตรวจสอบข้อมูลนำเข้า (Input Validation)	ไม่มีการตรวจสอบข้อมูลนำเข้า	เพิ่มการตรวจสอบข้อมูลนำเข้าผ่านฟังก์ชัน clean() และ validators ใน Django	Django Framework

การทดสอบระบบ (Software Testing)	ไม่มีการทดสอบอัตโนมัติ ใช้การทดสอบด้วยตนเอง	เพิ่มการทดสอบอัตโนมัติแบบ Unit Test, Integration Test และ End-to-End Test	Pytest, Robot Framework
การประเมินคุณภาพซอร์สโค้ด (Code Quality Evaluation)	ไม่ได้ทำการประเมินคุณภาพของโค้ด	ประเมินคุณภาพซอร์สโค้ดด้วย SonarQube โดยพิจารณา Reliability, Maintainability, Coverage และ Duplications	SonarQube

หมายเหตุ ระบบต้นแบบที่พัฒนาขึ้นในงานวิจัยนี้ใช้เพื่อจำลองกระบวนการของระบบสารสนเทศทั่วไป เพื่อประเมินประสิทธิภาพของแนวทางการปรับปรุงคุณภาพระบบ

จากตารางที่ 1.1 สามารถสรุปได้ว่า การเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติภายในระบบต้นแบบจะช่วยเพิ่มคุณภาพของระบบในหลายมิติ โดยเฉพาะด้านความถูกต้องของข้อมูล (Data Accuracy) ความเสถียรในการทำงาน (System Reliability) และความสามารถในการบำรุงรักษา (Maintainability)

1.6 คำสำคัญของการวิจัย

1. การทดสอบแบบอัตโนมัติ (Automated Testing) หมายถึง กระบวนการทดสอบซอฟต์แวร์โดยใช้เครื่องมือหรือสคริปต์ในการดำเนินการแทนการทดสอบด้วยตนเอง ช่วยลดความผิดพลาดจากมนุษย์และเพิ่มความรวดเร็วในการตรวจสอบคุณภาพของระบบ (Thoughtworks, 2012)

2. การตรวจสอบข้อมูลนำเข้า (Input Validation) หมายถึง กระบวนการตรวจสอบความถูกต้องของข้อมูลที่ผู้ใช้ป้อนเข้าสู่ระบบ เพื่อให้แน่ใจว่าข้อมูลมีรูปแบบและค่าที่ถูกต้องตามที่ระบบกำหนด ซึ่งช่วยลดข้อผิดพลาดของข้อมูลและป้องกันการโจมตีทางไซเบอร์ (O. Foundation, 2023)

3. Django Framework หมายถึง เฟรมเวิร์กโอเพนซอร์สที่พัฒนาด้วยภาษา Python ใช้สถาปัตยกรรมแบบ Model–View–Template (MVT) ซึ่งช่วยให้พัฒนาระบบเว็บแอปพลิเคชันได้อย่างเป็นระเบียบ ปลอดภัย และรองรับการทดสอบได้อย่างมีประสิทธิภาพ (Django Software, 2024)

4. การประเมินคุณภาพซอร์สโค้ด (Source Code Quality Evaluation) หมายถึง กระบวนการตรวจสอบและวิเคราะห์ซอร์สโค้ดเพื่อวัดคุณภาพของซอฟต์แวร์ในมิติที่สำคัญ เช่น ความถูกต้อง ความสามารถในการบำรุงรักษา และความครอบคลุมของการทดสอบ โดยใช้เครื่องมือวิเคราะห์เชิงสถิติ เช่น SonarQube (Humble & Farley, 2010)

5. SonarQube หมายถึง เครื่องมือวิเคราะห์คุณภาพซอร์สโค้ดแบบ Static Analysis ที่สามารถตรวจจับ Reliability, Maintainability, Coverage และ Duplications เพื่อใช้ประเมินคุณภาพของระบบซอฟต์แวร์ (SonarSource, 2024)

1.7 ประโยชน์ที่คาดว่าจะได้รับ

ได้แนวทางเชิงประยุกต์สำหรับการปรับปรุงคุณภาพของระบบสารสนเทศ จากการทดลองเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ ซึ่งสามารถนำไปใช้เป็นแนวทางในการพัฒนาหรือบำรุงรักษาระบบสารสนเทศในองค์กรให้มีความถูกต้องและมีประสิทธิภาพ

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในบทนี้ผู้วิจัยได้รวบรวมแนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้องซึ่งเป็นพื้นฐานในการพัฒนาแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ โดยมุ่งเน้นการศึกษากระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Dynamic Testing) เพื่อสร้างระบบต้นแบบสำหรับการทดลองและประเมินผลแนวทางดังกล่าว ทั้งนี้ได้ศึกษาทฤษฎีที่เกี่ยวข้องกับเครื่องมือและเทคโนโลยีที่ใช้ในงาน ได้แก่ Django Framework, Pytest, Robot Framework และ SonarQube เพื่อใช้เป็นแนวทางในการดำเนินการวิจัย

2.1 ทฤษฎีที่เกี่ยวข้อง

การศึกษาทฤษฎีในส่วนนี้มุ่งเน้นที่แนวคิดเกี่ยวกับการทดสอบซอฟต์แวร์และการประเมินคุณภาพระบบสารสนเทศ รวมถึงเทคโนโลยีที่เกี่ยวข้องกับ Django Framework และเครื่องมือที่ใช้ในการทดสอบและวิเคราะห์คุณภาพของซอร์สโค้ด

2.1.1 การทดสอบซอฟต์แวร์ (Software Testing)

การทดสอบซอฟต์แวร์ (Software Testing) เป็นกระบวนการที่สำคัญในวงจรการพัฒนาซอฟต์แวร์ (Software Development Life Cycle: SDLC) โดยมีจุดมุ่งหมายเพื่อให้มั่นใจว่าระบบที่พัฒนาขึ้นสามารถทำงานได้ตรงตามความต้องการของผู้ใช้และไม่มีข้อผิดพลาด ที่ส่งผลต่อประสิทธิภาพของระบบ (I. Foundation, 2023) กระบวนการทดสอบช่วยยืนยันความถูกต้องของระบบ ตรวจสอบพฤติกรรมของโปรแกรม และประเมินคุณภาพโดยรวมของซอฟต์แวร์ก่อนนำไปใช้งานจริง ตามหลักของ ISTQB (International Software Testing Qualifications Board) การทดสอบซอฟต์แวร์แบ่งออกได้หลายระดับตามลำดับขั้นของการพัฒนา ได้แก่ Unit Testing การทดสอบระดับหน่วยย่อยที่สุดของโปรแกรม Integration Testing การทดสอบการทำงานร่วมกันของโมดูลต่าง ๆ เพื่อให้มั่นใจว่าการเชื่อมโยงระหว่างส่วนประกอบทำงานถูกต้อง System Testing การทดสอบระบบโดยรวม เพื่อยืนยันว่าระบบทำงานได้ตรงตามข้อกำหนดที่กำหนดไว้ และ User Acceptance Testing การทดสอบโดยผู้ใช้ปลายทาง เพื่อยืนยันว่าระบบสามารถตอบสนองความต้องการของผู้ใช้ได้จริง

นอกจากนี้ การทดสอบซอฟต์แวร์ยังสามารถแบ่งตามลักษณะของการดำเนินการได้เป็นสองรูปแบบหลัก คือ Static Testing และ Dynamic Testing ซึ่งทั้งสองแบบมีเป้าหมายต่างกัน โดย Static Testing มุ่งเน้นการตรวจสอบคุณภาพของโค้ดหรือเอกสารโดยไม่ต้องรันระบบ ส่วน

Dynamic Testing เป็นการรันระบบจริงเพื่อตรวจสอบผลลัพธ์และพฤติกรรมของระบบภายใต้เงื่อนไขต่าง ๆ (I. Foundation, 2023)

ในปัจจุบัน องค์กรซอฟต์แวร์ขนาดใหญ่ได้ให้ความสำคัญกับ Automated Testing ซึ่งเป็นกระบวนการที่ใช้เครื่องมืออัตโนมัติในการรันชุดทดสอบ เพื่อเพิ่มความเร็วและความแม่นยำของการตรวจสอบ โดยเฉพาะอย่างยิ่งในระบบที่ต้องมีการพัฒนาและทดสอบซ้ำบ่อยครั้ง (Thoughtworks, 2012) การทดสอบที่มีประสิทธิภาพจะช่วยลดค่าใช้จ่ายในการบำรุงรักษาระบบ ลดข้อผิดพลาดที่อาจเกิดขึ้นหลังการใช้งานจริง และเพิ่มความน่าเชื่อถือของซอฟต์แวร์โดยรวม

กล่าวโดยสรุป การทดสอบซอฟต์แวร์จึงเป็นขั้นตอนสำคัญที่ไม่เพียงช่วยยืนยันความถูกต้องของระบบ แต่ยังเป็นกระบวนการที่สนับสนุนการพัฒนาอย่างต่อเนื่อง เพื่อให้ระบบสารสนเทศที่สร้างขึ้นมีคุณภาพ มีเสถียรภาพ และสามารถใช้งานได้อย่างมั่นใจ

2.1.2 แนวคิด Testing Pyramid

แนวคิด Testing Pyramid เป็นกรอบแนวคิดสำคัญในกระบวนการทดสอบซอฟต์แวร์ (Software Testing) ที่เสนอโดย (Cohn, 2010) ซึ่งอธิบายถึงสัดส่วนและความสัมพันธ์ระหว่างการทดสอบในแต่ละระดับ เพื่อให้การทดสอบมีประสิทธิภาพ ครอบคลุม และคุ้มค่าต่อเวลาและทรัพยากรที่ใช้ โดย Testing Pyramid มีลักษณะเป็นพีระมิดสามชั้น แบ่งตามระดับของการทดสอบดังนี้

1. Unit Test (ฐานพีระมิด) เป็นการทดสอบหน่วยย่อยที่สุดของโปรแกรม เช่น ฟังก์ชันหรือคลาส เพื่อให้มั่นใจว่าการทำงานของแต่ละส่วนเป็นไปตามที่ออกแบบไว้ การทดสอบระดับนี้ควรมีจำนวนมากที่สุด เนื่องจากรันได้รวดเร็ว มีต้นทุนต่ำ และสามารถตรวจจับข้อผิดพลาดได้ตั้งแต่ระยะเริ่มต้นของการพัฒนา

2. Integration Test (ชั้นกลาง) เป็นการทดสอบการทำงานร่วมกันของหลายโมดูลหรือหลายฟังก์ชัน เพื่อให้แน่ใจว่าการเชื่อมโยงข้อมูลและตรรกะระหว่างส่วนประกอบต่าง ๆ ของระบบถูกต้อง ตัวอย่างเช่น การตรวจสอบว่าฟังก์ชัน “ลงทะเบียนเรียน” สามารถเชื่อมโยงข้อมูลนักศึกษาและรายวิชาได้ถูกต้องระหว่างฐานข้อมูลและหน้าเว็บ

3. End-to-End Test (ยอดพีระมิด) เป็นการทดสอบกระบวนการทำงานของระบบทั้งหมดตั้งแต่ต้นจนจบ (System-Level Testing) โดยจำลองพฤติกรรมของผู้ใช้จริง เช่น การเข้าสู่ระบบ การเพิ่มหรือลบรายวิชา และการตรวจสอบผลการลงทะเบียน การทดสอบระดับนี้มีจำนวนไม่มากแต่ใช้เวลาและทรัพยากรสูงที่สุด

การจัดสัดส่วนการทดสอบตามแนวคิด Testing Pyramid ช่วยให้ทีมพัฒนาสามารถวางแผนและดำเนินการทดสอบได้อย่างเหมาะสม โดยเน้นการทดสอบระดับล่างให้มาก เพื่อค้นหาข้อผิดพลาดได้เร็วและมีค่าใช้จ่ายในการแก้ไขที่ต่ำ ในขณะที่การทดสอบระดับบนใช้เพื่อยืนยันความถูกต้องของระบบในมุมมองของผู้ใช้จริง (Fowler, 2019)

ในงานวิจัยนี้ แนวคิด Testing Pyramid ถูกนำมาใช้เป็นกรอบในการออกแบบและวางแผนการทดสอบของระบบลงทะเบียนเรียนต้นแบบ โดยมีการใช้ Pytest สำหรับการทดสอบระดับ Unit และ Integration และใช้ Robot Framework สำหรับการทดสอบระดับ End-to-End การดำเนินการตามแนวคิดดังกล่าวทำให้การประเมินคุณภาพของระบบมีความครอบคลุมทั้งในเชิงโครงสร้างและเชิงการใช้งานจริง ช่วยเพิ่มความน่าเชื่อถือและความถูกต้องของผลการทดสอบได้อย่างมีประสิทธิภาพ



รูป 2.1 Testing Pyramid

2.1.3 การทดสอบแบบสถิต (Static Testing)

การทดสอบแบบสถิต (Static Testing) เป็นกระบวนการตรวจสอบคุณภาพของซอฟต์แวร์โดยไม่จำเป็นต้องรันโปรแกรมจริง (I. Foundation, 2023) จุดประสงค์ของการทดสอบลักษณะนี้คือการค้นหาข้อผิดพลาด ที่อาจเกิดขึ้นในซอร์สโค้ดหรือเอกสารประกอบระบบตั้งแต่ระยะเริ่มต้นของการพัฒนา เพื่อลดโอกาสเกิดปัญหาในระหว่างการรันระบบจริงในภายหลัง การทดสอบแบบสถิตถือเป็นแนวทางที่ช่วยเพิ่มคุณภาพของซอฟต์แวร์และลดต้นทุนของการแก้ไขข้อผิดพลาดในระยะหลังได้อย่างมีประสิทธิภาพ รูปแบบของ Static Testing สามารถแบ่งออกเป็นสองประเภทหลัก ได้แก่

1. การตรวจสอบโดยมนุษย์ ซึ่งมักทำในลักษณะของการตรวจเอกสาร การตรวจโค้ดร่วมกัน (Code Review) หรือการตรวจสอบแบบเดินตามขั้นตอน (Walkthrough) เพื่อหาความผิดพลาดในตรรกะหรือมาตรฐานของการเขียนโปรแกรม

2. การวิเคราะห์ซอร์สโค้ดแบบอัตโนมัติ ใช้เครื่องมือช่วยตรวจสอบโครงสร้างและคุณภาพของโค้ดโดยอัตโนมัติ เช่น การตรวจจับข้อผิดพลาดในรูปแบบของ Syntax, การใช้ตัวแปรที่ไม่ได้ประกาศ, การซ้ำซ้อนของโค้ด (Duplications) หรือการตั้งชื่อที่ไม่เป็นไปตามมาตรฐาน

เครื่องมือยอดนิยมที่ใช้ในการวิเคราะห์คุณภาพของซอร์สโค้ดคือ SonarQube ซึ่งเป็นแพลตฟอร์มวิเคราะห์โค้ดแบบ Static Analysis ที่ช่วยให้ผู้พัฒนาสามารถตรวจสอบคุณภาพของซอฟต์แวร์ในหลายมิติ โดยเฉพาะมิติหลัก ๆ ดังต่อไปนี้

1. Reliability ใช้แทนตัวชี้วัดเดิม “Bugs” สำหรับประเมินระดับความน่าเชื่อถือของระบบ (Reliability Rating) โดยอิงจากจำนวนข้อผิดพลาดเชิงตรรกะหรือการทำงานที่อาจทำให้ระบบล้มเหลว การมีค่า Reliability ต่ำแสดงว่าระบบมีความเสถียรและมีข้อผิดพลาดน้อย

2. Maintainability ใช้แทนตัวชี้วัด “Code Smells” เพื่อประเมินความสามารถในการบำรุงรักษา (Maintainability Rating) ของซอร์สโค้ด โดยพิจารณาจากโครงสร้าง ความซ้ำซ้อน และการจัดรูปแบบของโค้ด การมีค่า Maintainability ต่ำบ่งชี้ว่าซอร์สโค้ดมีคุณภาพสูงและสามารถดูแลได้ง่าย

3. Coverage การวัดสัดส่วนของโค้ดที่ถูกครอบคลุมโดยกระบวนการทดสอบอัตโนมัติ (Automated Test Coverage) ซึ่งสะท้อนถึงความครอบคลุมของการทดสอบระบบ ค่า Coverage ที่สูงบ่งชี้ว่าระบบได้รับการทดสอบในหลายกรณีการทำงานและมีความน่าเชื่อถือมากขึ้น

4. Duplications การวัดอัตราส่วนของโค้ดที่มีการซ้ำซ้อนภายในระบบ ค่าที่ลดลงแสดงถึงการปรับปรุงประสิทธิภาพและคุณภาพของการออกแบบซอร์สโค้ดให้มีความเป็นมาตรฐานมากขึ้น

SonarQube ยังสนับสนุนการประเมินคุณภาพของซอฟต์แวร์ในมุมมองด้าน Reliability, Maintainability และ Security โดยแสดงผลการวิเคราะห์ในรูปแบบรายงานภาพรวมเพื่อช่วยให้นักพัฒนาและทีมทดสอบสามารถติดตามคุณภาพของโค้ดได้ต่อเนื่อง (SonarSource, 2024) เครื่องมือนี้ถูกนำมาใช้ในงานวิจัยและการพัฒนาเชิงอุตสาหกรรมอย่างแพร่หลาย เนื่องจากสามารถผสานเข้ากับเครื่องมือพัฒนาและระบบควบคุมเวอร์ชันได้โดยตรง

โดยสรุป การทดสอบแบบสถิตมีบทบาทสำคัญในการตรวจสอบคุณภาพของซอฟต์แวร์ก่อนเข้าสู่กระบวนการรันระบบจริงในขั้นตอนของ Dynamic Testing ช่วยให้ทีมพัฒนาสามารถระบุและแก้ไขปัญหาได้ตั้งแต่ระยะต้นของวงจรการพัฒนา ซึ่งนำไปสู่การเพิ่มประสิทธิภาพและลดความเสี่ยงในกระบวนการพัฒนาระบบโดยรวม

2.1.4 การทดสอบแบบพลวัต (Dynamic Testing)

การทดสอบแบบพลวัต (Dynamic Testing) เป็นกระบวนการทดสอบซอฟต์แวร์โดยการรันโปรแกรมจริง เพื่อสังเกตผลลัพธ์ที่เกิดขึ้นจากการทำงานของระบบภายใต้เงื่อนไขที่กำหนด (I. Foundation, 2023) จุดประสงค์หลักคือการตรวจสอบว่าซอฟต์แวร์สามารถทำงานได้อย่างถูกต้องตามที่ออกแบบไว้ และตอบสนองความต้องการของผู้ใช้ การทดสอบแบบพลวัตจึงเป็นการประเมินคุณภาพของซอฟต์แวร์ในสถานการณ์จริง โดยอาศัยข้อมูลนำเข้า และผลลัพธ์ที่ได้เพื่อประเมินความถูกต้องของระบบ การทดสอบแบบพลวัตสามารถแบ่งออกได้เป็นหลายระดับ ได้แก่

1. การทดสอบระดับหน่วย (Unit Testing) เป็นการทดสอบในระดับย่อยที่สุดของโปรแกรม เช่น ฟังก์ชัน หรือคลาส เพื่อให้แน่ใจว่าทำงานได้ถูกต้องตามที่ออกแบบไว้ โดยมักใช้เครื่องมือหรือ Framework เฉพาะ เช่น Pytest สำหรับภาษา Python ซึ่งสามารถสร้างและรันชุดทดสอบ (Test Cases) ได้อย่างอัตโนมัติและมีการรายงานผลการทดสอบอย่างละเอียด (Krekel, 2024)

2. การทดสอบแบบบูรณาการ (Integration Testing) เป็นการทดสอบการทำงานร่วมกันของโมดูลหรือฟังก์ชันต่าง ๆ ภายในระบบ เพื่อให้แน่ใจว่าการเชื่อมโยงข้อมูลระหว่างส่วนประกอบของระบบเป็นไปตามที่กำหนด เช่น การส่งข้อมูลจากหน้าแบบฟอร์ม ไปยังฐานข้อมูลผ่าน View และ Model ใน Django Framework

3. การทดสอบแบบ End-to-End Test (E2E Test) เป็นการทดสอบระบบโดยจำลองพฤติกรรมของผู้ใช้จริงตั้งแต่ต้นจนจบ เช่น การเข้าสู่ระบบ การลงทะเบียนเรียน และการบันทึกข้อมูล เพื่อให้แน่ใจว่าระบบทั้งหมดทำงานได้อย่างถูกต้องภายใต้สภาพแวดล้อมจริง โดยในโครงงานนี้ใช้ Robot Framework (Selenium) เป็นเครื่องมือหลักสำหรับดำเนินการทดสอบในลักษณะดังกล่าว (Robot Framework, 2024)

โดยสรุป การทดสอบแบบพลวัตจึงเป็นกระบวนการที่ช่วยให้สามารถประเมินคุณภาพของซอฟต์แวร์ได้อย่างแท้จริงจากการทำงานจริงของระบบ ช่วยเพิ่มความน่าเชื่อถือของระบบ ลดโอกาสการเกิดข้อผิดพลาด และทำให้ทีมพัฒนาสามารถปรับปรุงคุณภาพของซอฟต์แวร์ได้อย่างต่อเนื่องและมีประสิทธิภาพมากขึ้น

2.1.5 การทดสอบแบบอัตโนมัติ (Automated Testing)

การทดสอบแบบอัตโนมัติ (Automated Testing) คือกระบวนการทดสอบซอฟต์แวร์ที่ดำเนินการโดยใช้เครื่องมือหรือสคริปต์เพื่อรันการทดสอบแทนการทดสอบด้วยตนเอง โดยมีวัตถุประสงค์เพื่อเพิ่มความรวดเร็ว ความถูกต้อง และความสม่ำเสมอในการตรวจสอบคุณภาพของซอฟต์แวร์ (Thoughtworks, 2012)

แนวคิดของการทดสอบแบบอัตโนมัติเน้นที่การสร้างชุดการทดสอบ เพื่อใช้ตรวจสอบการทำงานของระบบในแต่ละส่วน โดยสามารถรันซ้ำได้ทุกครั้งที่มีการปรับปรุงโค้ด เพื่อให้แน่ใจว่าการแก้ไขหรือเพิ่มฟังก์ชันใหม่ไม่ได้ส่งผลกระทบต่อส่วนอื่นของระบบ (Minick, 2019) การนำเครื่องมืออัตโนมัติมาใช้ในการทดสอบจึงช่วยลดระยะเวลาในการตรวจสอบ เพิ่มความแม่นยำ และลดความผิดพลาดจากการทดสอบด้วยมนุษย์ เครื่องมือที่นิยมใช้ในกระบวนการทดสอบแบบอัตโนมัติแบ่งได้เป็นหลายประเภทตามระดับการทดสอบ ได้แก่

1. Pytest เป็นเครื่องมือสำหรับการทดสอบระดับหน่วย (Unit Test) และระดับบูรณาการ (Integration Test) ที่ได้รับความนิยมในภาษา Python เนื่องจากมีความยืดหยุ่นสูง สามารถเขียน Test Cases ได้ด้วยรูปแบบโค้ดที่เข้าใจง่าย รองรับการสร้างรายงานผลการทดสอบอัตโนมัติ

2. Robot Framework เป็นเครื่องมือสำหรับการทดสอบระดับ End-to-End Test (E2E Test) ที่ใช้แนวคิดแบบ Keyword-driven Testing Framework ซึ่งผู้ทดสอบสามารถเขียนคำสั่งการทดสอบในรูปแบบที่อ่านเข้าใจง่าย โดยมักใช้งานร่วมกับ Selenium เพื่อจำลองพฤติกรรมของผู้ใช้จริง เช่น การคลิก การกรอกข้อมูล และการตรวจสอบผลลัพธ์บนหน้าเว็บ (Robot Framework, 2024)

3. SonarQube แม้จะไม่ใช่เครื่องมือสำหรับรันการทดสอบโดยตรง แต่สามารถผสานเข้ากับระบบทดสอบอัตโนมัติได้ เพื่อใช้วิเคราะห์คุณภาพของซอร์สโค้ด (Static Analysis) และตรวจสอบความครอบคลุมของการทดสอบ (Coverage) ซึ่งช่วยให้การทดสอบอัตโนมัติมีประสิทธิภาพและได้ข้อมูลเชิงลึกมากยิ่งขึ้น (SonarSource, 2024)

โดยสรุป การทดสอบแบบอัตโนมัติเป็นแนวทางสำคัญที่ช่วยยกระดับคุณภาพของซอฟต์แวร์ ทำให้กระบวนการพัฒนาเป็นระบบมากขึ้น ลดข้อผิดพลาดจากมนุษย์ ที่มุ่งเน้นการส่งมอบระบบที่มีคุณภาพสูงอย่างต่อเนื่อง

2.1.6 Django Framework

Django Framework เฟรมเวิร์กแบบโอเพนซอร์ส (Open-source Framework) ที่พัฒนาด้วยภาษา Python ถูกออกแบบมาเพื่อช่วยให้นักพัฒนาสามารถสร้างเว็บแอปพลิเคชันได้อย่างรวดเร็ว มีโครงสร้างชัดเจน และมีระบบจัดการข้อมูลที่ปลอดภัย (Django Software, 2024) Django ใช้สถาปัตยกรรมแบบ Model–View–Template (MVT) ซึ่งช่วยแยกหน้าที่การทำงานของระบบออกจากกันอย่างชัดเจน ทำให้สามารถพัฒนา ทดสอบ และบำรุงรักษาได้อย่างมีประสิทธิภาพ

ในงานวิจัยนี้ Django Framework ถูกเลือกใช้เพื่อสร้าง ระบบต้นแบบจำลอง (Prototype System) สำหรับการทดลองแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ

เนื่องจากมีสถาปัตยกรรมที่รองรับการเพิ่ม Input Validation และสามารถเชื่อมต่อกับเครื่องมือทดสอบอัตโนมัติ เช่น Pytest และ Robot Framework ได้อย่างมีประสิทธิภาพ

2.1.7 การตรวจสอบข้อมูลนำเข้า (Input Validation)

การตรวจสอบข้อมูลนำเข้า (Input Validation) เป็นหนึ่งในกระบวนการของ Secure Software Development Lifecycle (SSDLC) ที่มุ่งลดความเสี่ยงจากข้อมูลที่ไม่ถูกต้องหรือมีเจตนาร้าย การออกแบบขั้นตอนนี้อย่างเหมาะสมเป็นการป้องกันช่องโหว่ระดับต้นน้ำ เช่น SQL Injection หรือ Cross-site Scripting (XSS) ซึ่งเป็นปัญหาที่พบบ่อยในระบบเว็บแอปพลิเคชัน (O. Foundation, 2023) การตรวจสอบข้อมูลนำเข้าเป็นกระบวนการสำคัญในระบบสารสนเทศที่รับข้อมูลจากผู้ใช้ เพื่อให้มั่นใจว่าข้อมูลที่ได้รับนั้นอยู่ในรูปแบบที่ถูกต้อง เหมาะสม และไม่ก่อให้เกิดข้อผิดพลาดในระบบ กระบวนการนี้ช่วยป้องกันไม่ให้ข้อมูลที่ไม่เป็นไปตามมาตรฐาน เช่น ตัวอักษรพิเศษหรือคำสั่งที่เป็นอันตราย ถูกส่งเข้าสู่ระบบ ซึ่งอาจนำไปสู่การโจมตีทางไซเบอร์ หลักการของ Input Validation สามารถแบ่งออกได้เป็น 2 ลักษณะหลัก คือ

1. Client-side Validation การตรวจสอบข้อมูลที่ฝั่งผู้ใช้ก่อนส่งข้อมูลเข้าสู่เซิร์ฟเวอร์ วิธีนี้ช่วยลดภาระของเซิร์ฟเวอร์และให้ประสบการณ์ที่ดีแก่ผู้ใช้ แต่ยังไม่สามารถป้องกันการโจมตีได้อย่างสมบูรณ์ เพราะผู้ไม่หวังดีสามารถข้ามการตรวจสอบฝั่ง Client ได้

2. Server-side Validation การตรวจสอบข้อมูลที่ฝั่งเซิร์ฟเวอร์ก่อนจัดเก็บลงฐานข้อมูล ถือเป็นกระบวนการสำคัญที่ต้องมีในทุกระบบ เนื่องจากเป็นการป้องกันขั้นสุดท้ายก่อนข้อมูลจะเข้าสู่ระบบหลัก

ใน Django Framework มีเครื่องมือสำหรับการตรวจสอบข้อมูลนำเข้าในหลายระดับ ทั้งในส่วนของ Form และ Model ซึ่งช่วยให้นักพัฒนาสามารถควบคุมรูปแบบของข้อมูลได้อย่างเป็นระบบ ตัวอย่างกลไกที่ใช้บ่อย ได้แก่

1. เมธอด clean() ใน Django Form ซึ่งสามารถใช้กำหนดเงื่อนไขการตรวจสอบข้อมูลเฉพาะฟิลด์ เช่น การตรวจสอบความยาวของข้อความ หรือรูปแบบของอีเมล

2. คลาส validators ที่ Django มีให้ใช้งาน เพื่อช่วยตรวจสอบค่าข้อมูลในรูปแบบต่าง ๆ เช่น ตัวเลข อีเมล หรือรหัสผ่านตามเงื่อนไขที่กำหนด

3. การใช้ Attribute เช่น max_length, required, และ unique ใน Model เพื่อควบคุมความถูกต้องของข้อมูลในระดับฐานข้อมูล

การใช้ Input Validation ไม่เพียงช่วยป้องกันข้อผิดพลาดจากผู้ใช้เท่านั้น แต่ยังเป็นแนวทางปฏิบัติที่สำคัญในการรักษาความปลอดภัยของระบบ โดย OWASP (Open Worldwide Application Security Project) ได้จัดให้ “การตรวจสอบข้อมูลนำเข้า” เป็นหนึ่งในแนวทางหลักในการพัฒนาเว็บแอปพลิเคชันที่ปลอดภัย (O. Foundation, 2023)

ในบริบทของงานวิจัยนี้ การเพิ่มกระบวนการ Input Validation ภายในระบบลงทะเบียนเรียนที่พัฒนาด้วย Django Framework มีเป้าหมายเพื่อเพิ่มความถูกต้องของข้อมูล ลดความผิดพลาดจากการกรอกข้อมูลของผู้ใช้ และลดความเสี่ยงจากช่องโหว่ด้านความปลอดภัย การดำเนินการดังกล่าวยังช่วยให้ระบบมีความเสถียรมากขึ้น และมีประสิทธิภาพในด้านความปลอดภัยของข้อมูล

2.1.8 Pytest

Pytest เป็นเฟรมเวิร์กสำหรับการทดสอบซอฟต์แวร์ในภาษา Python ที่ได้รับความนิยมอย่างแพร่หลาย เนื่องจากมีความยืดหยุ่น ใช้งานง่าย และสามารถขยายฟังก์ชันการทำงานได้หลากหลาย โดยเฉพาะในการพัฒนาระบบด้วย Django Framework ที่ต้องการการทดสอบทั้งในระดับหน่วยย่อย (Unit Test) และระดับรวม (Integration Test) (Krekel et al., 2023)

จุดเด่นของ Pytest คือการรองรับโครงสร้างการทดสอบแบบ ฟังก์ชัน (Function-Based Testing) และการใช้ Fixture เพื่อจำลองข้อมูลหรือสภาพแวดล้อมที่ต้องใช้ซ้ำในหลายกรณี ทำให้โค้ดทดสอบกระชับ อ่านง่าย และสามารถบำรุงรักษาได้ง่าย นอกจากนี้ยังรองรับการประยุกต์ใช้ร่วมกับปลั๊กอินต่าง ๆ เช่น pytest-django สำหรับการทดสอบแอปพลิเคชัน Django โดยเชื่อมต่อกับฐานข้อมูลจำลอง pytest-cov สำหรับการสร้างรายงาน Test Coverage เพื่อวัดสัดส่วนของโค้ดที่ถูกทดสอบ pytest-xdist สำหรับการรันทดสอบแบบขนาน (parallel testing) เพื่อประหยัดเวลา

การทำงานของ Pytest ประกอบด้วยขั้นตอนสำคัญ 3 ขั้นตอน คือ

1. การจัดเตรียมสภาพแวดล้อม (Arrange) สร้างวัตถุหรือข้อมูลจำลองที่จำเป็นสำหรับการทดสอบ

2. การกระทำ (Act) เรียกใช้ฟังก์ชันหรือเมธอดที่ต้องการทดสอบ

3. การตรวจสอบผลลัพธ์ (Assert) ตรวจสอบว่าผลลัพธ์ที่ได้ตรงตามที่คาดหมายหรือไม่

ในการดำเนินการวิจัยครั้งนี้ Pytest ถูกนำมาใช้สำหรับการทดสอบระดับ Unit Test และ Integration Test ของระบบลงทะเบียนเรียนต้นแบบ เพื่อให้มั่นใจว่าฟังก์ชันแต่ละส่วน เช่น การเพิ่มรายวิชา การตรวจสอบเวลาลงทะเบียน และการคำนวณหน่วยกิต ทำงานถูกต้องตามที่ออกแบบไว้ ผลการทดสอบจาก Pytest ยังถูกนำไปวิเคราะห์ต่อใน SonarQube เพื่อประเมิน Test Coverage และความน่าเชื่อถือ (Reliability) ของระบบซอฟต์แวร์โดยรวม

ดังนั้น Pytest จึงมีบทบาทสำคัญในการยืนยันคุณภาพของซอฟต์แวร์ และเป็นพื้นฐานของการพัฒนา ซึ่งช่วยให้การพัฒนาระบบเป็นไปอย่างมีประสิทธิภาพและลดข้อผิดพลาดในระยะยาว

2.1.9 Robot Framework

Robot Framework เป็นเฟรมเวิร์กสำหรับการทดสอบแบบอัตโนมัติ (Automated Testing Framework) ที่พัฒนาด้วยภาษา Python และได้รับความนิยมอย่างแพร่หลายในงานทดสอบเชิงระบบ (System Testing) และงานทดสอบแบบ End-to-End (E2E Testing) เนื่องจากมีโครงสร้างที่อ่านเข้าใจง่าย ใช้คำสั่งในลักษณะ “Keyword-Driven” ทำให้ทั้งนักพัฒนาและผู้ที่ไม่ใช่นักโปรแกรมสามารถเข้าใจสคริปต์ทดสอบได้ (Robot Framework, 2024)

Robot Framework ถูกออกแบบมาให้รองรับการทดสอบซอฟต์แวร์ในหลายมิติ ไม่ว่าจะเป็นการทดสอบเว็บแอปพลิเคชัน อินเทอร์เฟซของโปรแกรม (API) หรือแม้แต่ระบบเชิงกระบวนการ (Process Automation) โดยสามารถขยายความสามารถผ่านไลบรารีต่าง ๆ ได้ เช่น SeleniumLibrary สำหรับการทดสอบการทำงานของเว็บผ่านเบราว์เซอร์จริง RequestsLibrary สำหรับทดสอบ REST API DatabaseLibrary สำหรับเชื่อมต่อและตรวจสอบฐานข้อมูล

Robot Framework ใช้ไฟล์ทดสอบที่มีนามสกุล .robot ซึ่งสามารถเขียนได้ในรูปแบบที่ใกล้เคียงภาษามนุษย์

ในการดำเนินงานวิจัยครั้งนี้ Robot Framework ถูกใช้เพื่อสร้างชุดการทดสอบแบบ End-to-End (E2E Testing) สำหรับระบบลงทะเบียนเรียนต้นแบบที่พัฒนาใน Django โดยชุดการทดสอบจะจำลองพฤติกรรมของผู้ใช้จริง ตั้งแต่การเข้าสู่ระบบ การเพิ่ม–ลบรายวิชา การตรวจสอบตารางเรียน ไปจนถึงการบันทึกผลลัพธ์การลงทะเบียน

การใช้ Robot Framework มีข้อดีคือสามารถทำให้การทดสอบระบบทำได้ซ้ำได้อย่างอัตโนมัติ ลดเวลาการทดสอบเมื่อเทียบกับการทดสอบแบบแมนนวล และยังช่วยให้สามารถสร้างรายงานผลการทดสอบ (Test Report) ในรูปแบบ HTML ได้โดยอัตโนมัติ ซึ่งแสดงผลการทดสอบในแต่ละขั้นตอนอย่างละเอียด

ผลการทดสอบจาก Robot Framework ถูกนำมาวิเคราะห์ร่วมกับผลการทดสอบจาก Pytest เพื่อให้การประเมินระบบมีความครอบคลุมทั้งในระดับ ฟังก์ชันย่อย (Unit Level) และกระบวนการทำงานจริง (System Level) สอดคล้องกับแนวคิด Testing Pyramid ที่เน้นการทดสอบในหลายระดับ เพื่อเพิ่มความน่าเชื่อถือของระบบโดยรวม

ดังนั้น การใช้ Robot Framework ในการวิจัยนี้ไม่เพียงแต่ช่วยลดเวลาและความผิดพลาดจากการทดสอบด้วยมือเท่านั้น แต่ยังเพิ่มความสามารถในการตรวจสอบความถูกต้องของระบบลงทะเบียนเรียนในมุมมองของผู้ใช้งานจริง ทำให้ผลการประเมินคุณภาพของระบบมีความสมบูรณ์และครอบคลุมมากยิ่งขึ้น

2.1.10 SonarQube

SonarQube เป็นเครื่องมือสำหรับการวิเคราะห์คุณภาพของซอร์สโค้ดแบบสถิต (Static Code Analysis) ที่ได้รับการพัฒนาโดยบริษัท SonarSource ซึ่งมีจุดประสงค์เพื่อช่วยให้นักพัฒนาสามารถตรวจสอบคุณภาพของซอฟต์แวร์ได้อย่างเป็นระบบ โดยไม่จำเป็นต้องรันโปรแกรมจริง (SonarSource, 2024) เครื่องมือนี้สามารถตรวจจับข้อผิดพลาดเชิงตรรกะ (Bugs) ช่องโหว่ด้านความปลอดภัย (Vulnerabilities) และโค้ดที่มีลักษณะซับซ้อนเกินไปหรือเข้าใจยาก (Code Smells) ได้อย่างอัตโนมัติ

การทำงานของ SonarQube แบ่งออกเป็น 3 ขั้นตอนหลัก ได้แก่

1. การสแกนซอร์สโค้ด (Code Scanning) เครื่องมือ SonarScanner จะทำการอ่านซอร์สโค้ดของโครงการและตรวจสอบตามกฎ (Rules) ที่ตั้งค่าไว้ในระบบ SonarQube โดยกฎเหล่านี้ครอบคลุมประเด็นด้าน Reliability, Security, และ Maintainability

2. การวิเคราะห์และคำนวณคะแนนคุณภาพ (Quality Analysis) ระบบจะวิเคราะห์โค้ดตามผลการสแกน พร้อมคำนวณคะแนนในรูปแบบ Quality Metrics เช่น Reliability Rating, Maintainability Rating, Security Rating และ Test Coverage

3. การรายงานผลและแสดงภาพรวม (Reporting) SonarQube แสดงผลการวิเคราะห์ผ่าน Dashboard แบบกราฟิก ซึ่งช่วยให้นักพัฒนามองเห็นจุดที่ต้องปรับปรุงได้ทันที

SonarQube ใช้แนวคิดการประเมินคุณภาพซอฟต์แวร์แบบ “Quality Gate” เพื่อกำหนดเกณฑ์ขั้นต่ำของคุณภาพ หากโครงการไม่ผ่านเงื่อนไข เช่น มี Bug หรือ Vulnerability เกินจำนวนที่กำหนด ระบบจะถือว่า “ไม่ผ่าน” (Failed) ทำให้ผู้พัฒนาทราบว่าต้องปรับปรุงก่อนจะเผยแพร่ซอฟต์แวร์ต่อไป

ในการดำเนินการวิจัยครั้งนี้ SonarQube ถูกใช้เป็นเครื่องมือวิเคราะห์ซอร์สโค้ดของระบบลงทะเบียนเรียนทั้งสองเวอร์ชัน ผลการวิเคราะห์จาก SonarQube แสดงให้เห็นความแตกต่างอย่างชัดเจนในด้านคุณภาพของซอฟต์แวร์ โดยเฉพาะค่าดัชนี Reliability, Maintainability และ Test Coverage ซึ่งใช้เป็นตัวชี้วัดประสิทธิภาพของระบบหลังการปรับปรุง นอกจากนี้ SonarQube ยังรองรับการเชื่อมต่อกับระบบ Continuous Integration (CI) และ Continuous Deployment (CD) เช่น Jenkins หรือ GitHub Actions ซึ่งช่วยให้งานวิเคราะห์โค้ดสามารถดำเนินการได้อัตโนมัติทุกครั้งที่มีการอัปเดตซอร์สโค้ด ทำให้ผลการประเมินมีความต่อเนื่องและเป็นปัจจุบันอยู่เสมอ

SonarQube ทำหน้าที่เป็นเครื่องมือหลักในการประเมินคุณภาพของซอฟต์แวร์เชิงปริมาณในงานวิจัยนี้ โดยเป็นการวิเคราะห์แบบไม่ต้องรันโปรแกรม (Static Testing) ซึ่งช่วยตรวจจับจุดบกพร่องในโค้ดได้รวดเร็วและครอบคลุมมากกว่าเดิม ผลการวิเคราะห์ยังสามารถนำมาใช้เปรียบเทียบความแตกต่างระหว่างระบบเดิมและระบบใหม่ได้อย่างเป็นรูปธรรม ส่งผลให้การประเมิน

ประสิทธิภาพของระบบหลังการปรับปรุงมีความน่าเชื่อถือ และสามารถยืนยันได้ว่าการเพิ่มกลไก Input Validation และ Automated Testing ช่วยยกระดับคุณภาพของซอฟต์แวร์ได้จริง

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 งานวิจัยด้านการตรวจสอบข้อมูลนำเข้า (Input Validation)

งานของ (Faisal & Elshoush, 2023) ได้ทำการศึกษาทบทวนอย่างเป็นระบบเกี่ยวกับช่องโหว่จากการตรวจสอบข้อมูลนำเข้าในเว็บแอปพลิเคชัน โดยได้จำแนกประเภทของช่องโหว่ เทคนิคการโจมตีที่เกิดจากการไม่ตรวจสอบข้อมูล และแนวทางแก้ไขที่เป็นปัจจุบัน พบว่าการกำหนดกระบวนการตรวจสอบข้อมูลตั้งแต่ชั้นรับข้อมูล (input layer) สามารถลดโอกาสการเกิดข้อผิดพลาดและการโจมตีทางไซเบอร์ได้อย่างมีนัยสำคัญ งานวิจัยนี้จึงเป็นหลักฐานเชิงประจักษ์ที่ยืนยันว่าการเพิ่มกลไก Input Validation มีส่วนช่วยเพิ่มความปลอดภัยและความถูกต้องของข้อมูลในระบบสารสนเทศได้อย่างมีประสิทธิภาพ

นอกจากนี้ (O. Foundation, 2023) ได้จัดทำคู่มือมาตรฐานสำหรับการตรวจสอบข้อมูลนำเข้า ซึ่งระบุแนวทางที่เหมาะสมในการออกแบบฟังก์ชันตรวจสอบข้อมูลในเว็บแอปพลิเคชัน เช่น การตรวจสอบประเภทข้อมูล การจำกัดขนาดข้อมูล และการใช้ตัวกรองที่สอดคล้องกับมาตรฐานความปลอดภัยสากล (OWASP Top 10) คู่มือนี้ได้รับการอ้างอิงอย่างแพร่หลายในงานพัฒนาเว็บแอปพลิเคชันสมัยใหม่ และสามารถประยุกต์ใช้ได้โดยตรงกับระบบที่พัฒนาด้วย Django Framework ซึ่งรองรับฟังก์ชัน validators และ clean() สำหรับตรวจสอบข้อมูลก่อนบันทึกลงฐานข้อมูล

2.2.2 งานวิจัยด้านการทดสอบแบบอัตโนมัติ (Automated Testing)

งานของ ("Usage of Robot Framework in Automation of Functional Test," 2014) ได้นำเสนอแนวทางการใช้ Robot Framework ในการทำ Functional Test และ Regression Test อัตโนมัติในระบบเว็บแอปพลิเคชัน โดยใช้โครงสร้างแบบ Keyword-Driven Testing เพื่อให้สามารถเขียนสคริปต์ทดสอบได้อย่างเป็นระบบ ผลการทดลองแสดงให้เห็นว่าการใช้ Robot Framework ช่วยลดเวลาการทดสอบลงและเพิ่มความถูกต้องในการตรวจสอบการทำงานของระบบ งานนี้จึงสนับสนุนแนวคิดของการนำเครื่องมือทดสอบอัตโนมัติมาใช้ในงานวิจัยนี้ ซึ่งเลือกใช้ Robot Framework ร่วมกับ Selenium ในการทดสอบแบบ End-to-End (E2E Test)

2.2.3 งานวิจัยด้านการวิเคราะห์คุณภาพของซอร์สโค้ด (Static Code Analysis)

งานของ (Etemadi et al., 2021) ได้นำเสนอเครื่องมือ Sorald ซึ่งทำงานร่วมกับ SonarQube เพื่อเสนอแนวทางการแก้ไขปัญหาที่ตรวจพบในขั้นตอนการวิเคราะห์ซอร์สโค้ดอัตโนมัติ โดยเครื่องมือนี้สามารถสร้าง patch อัตโนมัติสำหรับ violations ที่ตรวจพบในโค้ด Java จาก SonarQube ซึ่งช่วยลดภาระของผู้พัฒนาและเพิ่มประสิทธิภาพในการปรับปรุงคุณภาพซอฟต์แวร์

งานนี้แสดงให้เห็นศักยภาพของ SonarQube ในการตรวจวัดและปรับปรุงคุณภาพของซอร์สโค้ดอย่างเป็นระบบ ซึ่งสอดคล้องกับแนวทางของงานวิจัยนี้ที่ใช้ SonarQube เป็นเครื่องมือในการวิเคราะห์ตัวชี้วัด Bugs, Code Smells และ Coverage เพื่อเปรียบเทียบประสิทธิภาพของระบบก่อนและหลังการปรับปรุง

บทที่ 3
วิธีดำเนินงานวิจัย

บทนี้อธิบายขั้นตอนและกระบวนการดำเนินงานวิจัยที่ใช้ในการพัฒนาระบบต้นแบบเพื่อปรับปรุงประสิทธิภาพของระบบสารสนเทศ ซึ่งประกอบด้วยการศึกษาและวิเคราะห์ทฤษฎีที่เกี่ยวข้อง การออกแบบและพัฒนาระบบต้นแบบด้วย Django Framework การเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Testing) ตลอดจนการประเมินคุณภาพของซอร์สโค้ดด้วยเครื่องมือ SonarQube เพื่อเปรียบเทียบประสิทธิภาพระหว่างระบบก่อนปรับปรุง (T0) และระบบหลังปรับปรุง (T1)

3.1 การศึกษาทฤษฎีที่เกี่ยวข้องกับงานวิจัย หรือ การศึกษาและวิเคราะห์ปัญหางานวิจัย

การดำเนินงานวิจัยเริ่มจากการศึกษาและวิเคราะห์ปัญหาที่เกิดขึ้นในระบบสารสนเทศทั่วไป โดยพบว่าระบบจำนวนมากยังขาดกลไกในการตรวจสอบข้อมูลนำเข้าและกระบวนการทดสอบที่เป็นระบบ ส่งผลให้เกิดข้อผิดพลาดระหว่างการทำงาน และขาดเครื่องมือประเมินคุณภาพซอร์สโค้ดที่เป็นรูปธรรม เพื่อแก้ปัญหาดังกล่าว ผู้วิจัยจึงได้ศึกษาแนวทางการเพิ่มประสิทธิภาพของระบบสารสนเทศโดยประยุกต์ใช้ Input Validation และ Automated Dynamic Testing

เพื่อทดสอบแนวทางที่เสนอ ผู้วิจัยได้สร้างระบบต้นแบบจำลอง (Prototype System) โดยใช้ Django Framework เป็นเครื่องมือหลักในการพัฒนา เพื่อจำลองการทำงานของระบบสารสนเทศทั่วไป ระบบต้นแบบดังกล่าวใช้เป็นกรณีศึกษาสำหรับการเพิ่มฟังก์ชันตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ รวมถึงใช้สำหรับการวัดผลด้วย SonarQube เพื่อเปรียบเทียบคุณภาพของระบบก่อนและหลังการปรับปรุง (T0 และ T1)

3.2 การรวบรวมและเตรียมข้อมูล

ข้อมูลที่ใช้ในการทดลองเป็น ข้อมูลจำลอง ซึ่งสร้างขึ้นเพื่อใช้ในระบบต้นแบบ โดยไม่มีการนำข้อมูลจริงของผู้ใช้มาใช้ในการทดสอบ เพื่อให้เป็นไปตามหลักจริยธรรมทางวิจัย ข้อมูลประกอบด้วย

1. ข้อมูลผู้ใช้ (User Data) ใช้ทดสอบฟังก์ชันการเข้าสู่ระบบ การลงทะเบียน และการตรวจสอบสิทธิ์ของผู้ใช้

2. ข้อมูลรายวิชา (Course Data) ใช้จำลองข้อมูลในตารางรายวิชา เช่น รหัสวิชา ชื่อรายวิชา หน่วยกิต และอาจารย์ผู้สอน

3. ข้อมูลการลงทะเบียนเรียน (Registration Data) ใช้ทดสอบการทำงานของระบบในส่วนการเชื่อมโยงข้อมูลระหว่างผู้ใช้และรายวิชา

ข้อมูลทั้งหมดถูกจัดเก็บในฐานข้อมูล PostgreSQL และจัดการผ่าน Django ORM (Object-Relational Mapping) เพื่อความสะดวกในการเข้าถึงและการเชื่อมโยงข้อมูลระหว่างโมดูลต่าง ๆ ของระบบ

3.3 เครื่องมือที่ใช้ในการทำวิจัยและขั้นตอนการทำวิจัย

3.3.1 เครื่องมือที่ใช้ในการทำวิจัย

ในการดำเนินงานวิจัยครั้งนี้ ได้ใช้เครื่องมือหลักในการพัฒนา ทดสอบ และประเมินผล ดังนี้

1) Django Framework

ใช้ในการพัฒนาระบบต้นแบบด้วยสถาปัตยกรรมแบบ Model–View–Template (MVT) เพื่อจำลองการทำงานของระบบสารสนเทศทั่วไป

2) PostgreSQL

ใช้เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (RDBMS) สำหรับจัดเก็บข้อมูลผู้ใช้ รายวิชา และข้อมูลการลงทะเบียนเรียนในระบบต้นแบบ

3) Pytest

ใช้สำหรับการทดสอบระดับหน่วย (Unit Test) และการทดสอบแบบบูรณาการ (Integration Test) เพื่อยืนยันความถูกต้องของฟังก์ชันภายในระบบ

4) Robot Framework (Selenium Library)

ใช้ในการทดสอบแบบ End-to-End (E2E) เพื่อจำลองพฤติกรรมการใช้งานของผู้ใช้จริงบนหน้าเว็บ

5) SonarQube

ใช้สำหรับการประเมินคุณภาพซอร์สโค้ด โดยวัดค่าตัวชี้วัดหลัก ได้แก่ Reliability, Maintainability, Coverage และ Duplications เพื่อเปรียบเทียบประสิทธิภาพของระบบก่อนและหลังปรับปรุง

6) Visual Studio Code

ใช้เป็นสภาพแวดล้อมในการพัฒนาและควบคุมเวอร์ชันของซอร์สโค้ด เพื่อให้การพัฒนาและทดสอบเป็นไปอย่างมีประสิทธิภาพ

7) Windows 10

ใช้เป็นระบบปฏิบัติการหลักในการพัฒนา ติดตั้ง และทดสอบระบบต้นแบบ เพื่อให้การทำงานของเครื่องมือทั้งหมดมีความเข้ากันได้

3.3.2 ขั้นตอนการทำวิจัย

1. ศึกษาเอกสารและทฤษฎีที่เกี่ยวข้องกับ Input Validation, Automated Testing และ SonarQube

2. ออกแบบระบบต้นแบบโดยใช้ Django Framework เพื่อจำลองระบบสารสนเทศทั่วไป และสร้างฐานข้อมูล

3. พัฒนา ระบบก่อนปรับปรุง (T0) โดยไม่มีการเพิ่มการตรวจสอบข้อมูลและการทดสอบอัตโนมัติ

4. พัฒนา ระบบหลังปรับปรุง (T1) โดยเพิ่ม Input Validation และการทดสอบอัตโนมัติในหลายระดับ (Unit, Integration, E2E)

5. ประเมินคุณภาพของระบบทั้งสองเวอร์ชันด้วย SonarQube เพื่อวัดค่า Reliability, Maintainability, Coverage และ Duplications

6. วิเคราะห์ผลการเปรียบเทียบและสรุปผลการปรับปรุงประสิทธิภาพของระบบสารสนเทศระหว่าง T0 และ T1

3.4 การออกแบบและพัฒนาระบบต้นแบบ

ระบบต้นแบบที่ใช้ในการวิจัยนี้ถูกออกแบบและพัฒนาโดยใช้ Django Framework ซึ่งเป็นเฟรมเวิร์กแบบโอเพนซอร์สที่พัฒนาด้วยภาษา Python มีสถาปัตยกรรมแบบ Model–View–Template (MVT) ช่วยแยกส่วนการทำงานของระบบออกจากกันอย่างชัดเจน เพื่อให้ง่ายต่อการพัฒนา การทดสอบ และการบำรุงรักษาในภายหลัง

โครงสร้างการออกแบบระบบต้นแบบแบ่งออกเป็น 3 ส่วนหลัก ดังนี้

1. Model ใช้สำหรับจัดการโครงสร้างข้อมูลของระบบ เช่น ตารางผู้ใช้ ตารางรายวิชา และข้อมูลการลงทะเบียนเรียน โดยเชื่อมโยงข้อมูลผ่าน Django ORM เพื่อควบคุมความสัมพันธ์ของข้อมูลภายในฐานข้อมูล PostgreSQL

2. View ทำหน้าที่ควบคุมการประมวลผลคำขอจากผู้ใช้ (Request) และตอบสนองผลลัพธ์ (Response) โดยมีการเชื่อมโยงกับส่วน Model และ Template เพื่อให้ระบบทำงานได้อย่างครบวงจร

3. Template ใช้สำหรับแสดงผลข้อมูลและส่วนติดต่อผู้ใช้ (User Interface) ผ่านหน้าเว็บ โดยออกแบบให้เรียบง่ายและเหมาะสมต่อการทดสอบพฤติกรรมของผู้ใช้จริง

เพื่อรองรับแนวทางการปรับปรุงคุณภาพของระบบ ระบบต้นแบบได้มีการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) ในระดับฟอร์มและโมเดล เพื่อควบคุมความถูกต้องของ

ข้อมูลก่อนจัดเก็บเข้าสู่ฐานข้อมูล เช่น การตรวจสอบรูปแบบอีเมล การป้องกันการกรอกข้อมูลซ้ำ และการจำกัดค่าข้อมูลตามเงื่อนไขที่กำหนด

นอกจากนี้ยังได้เพิ่มกระบวนการ ทดสอบแบบอัตโนมัติ (Automated Dynamic Testing) เพื่อประเมินประสิทธิภาพของระบบในหลายระดับ ดังนี้

1. Unit Test ใช้ Pytest ตรวจสอบการทำงานของฟังก์ชันย่อยแต่ละส่วนของระบบ

2. Integration Test ตรวจสอบการเชื่อมโยงข้อมูลและการทำงานร่วมกันระหว่างโมดูล

3. End-to-End Test (E2E Test) ใช้ Robot Framework (Selenium) จำลองพฤติกรรมผู้ใช้จริง เช่น การเข้าสู่ระบบ การลงทะเบียนรายวิชา และการตรวจสอบผลลัพธ์บนหน้าเว็บ

จากนั้นระบบทั้งสองเวอร์ชัน คือ ระบบก่อนปรับปรุง (T0) และ ระบบหลังปรับปรุง (T1) จะถูกนำไปประเมินคุณภาพด้วยเครื่องมือ SonarQube เพื่อวิเคราะห์คุณภาพของซอร์สโค้ดในเชิงสถิติ โดยพิจารณาตัวชี้วัดหลัก ได้แก่ Reliability, Maintainability, Coverage และ Duplications ซึ่งผลลัพธ์จะถูกนำไปเปรียบเทียบเพื่อวัดอัตราการปรับปรุงประสิทธิภาพของระบบ

3.5 วิธีการทดสอบและวิเคราะห์ผล

การทดสอบและวิเคราะห์ผลของงานวิจัยนี้มีวัตถุประสงค์เพื่อประเมินประสิทธิภาพของแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ โดยเปรียบเทียบผลลัพธ์ระหว่างระบบก่อนปรับปรุง (T0) และระบบหลังปรับปรุง (T1) เพื่อให้เห็นผลเชิงประจักษ์ของการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ โดยมีขั้นตอนการทดลองดังนี้

1. ระยะที่ 1 (T0: ระบบก่อนปรับปรุง) ทำการทดสอบระบบต้นแบบเดิมที่ยังไม่มีการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบอัตโนมัติ (Automated Testing) เพื่อเก็บข้อมูลพื้นฐานด้านคุณภาพซอร์สโค้ด โดยใช้เครื่องมือ SonarQube วิเคราะห์ค่าตัวชี้วัดหลัก ได้แก่ Reliability, Maintainability, Coverage และ Duplications ซึ่งผลลัพธ์จากระยะนี้ใช้เป็นค่ามาตรฐานอ้างอิงในการเปรียบเทียบ

2. ระยะที่ 2 (T1: ระบบหลังปรับปรุง) ทำการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าในส่วนของแบบฟอร์มของระบบ เพื่อควบคุมความถูกต้องและรูปแบบของข้อมูลก่อนจัดเก็บ และเพิ่มกระบวนการทดสอบอัตโนมัติในสามระดับ ได้แก่

1. Unit Testing ใช้ Pytest สำหรับการทดสอบฟังก์ชันภายในโมดูลของระบบ

2. Integration Testing ทดสอบการทำงานร่วมกันระหว่างโมดูล เช่น การเพิ่ม แก้ไข และบันทึกข้อมูล

3. End-to-End Testing ใช้ Robot Framework จำลองการทำงานของผู้ใช้จริงในระบบ เพื่อทดสอบการทำงานของระบบในภาพรวม

จากนั้นใช้เครื่องมือ SonarQube ทำการวิเคราะห์คุณภาพซอร์สโค้ดของระบบที่ปรับปรุงแล้ว (T1) โดยใช้ตัวชี้วัดเดียวกันกับระยะ T0 เพื่อให้การเปรียบเทียบมีความสอดคล้องกัน

3. ขอบเขตของซอร์สโค้ดที่นำไปวิเคราะห์ใน SonarQube การประเมินคุณภาพซอร์สโค้ดในแต่ละระยะ (T0 และ T1) ดำเนินการโดยใช้เครื่องมือ SonarQube วิเคราะห์ซอร์สโค้ดทั้งหมดของระบบที่พัฒนาใน Django Framework โดยรวมถึงไฟล์ในแต่ละโมดูลหลัก ดังนี้

1. models ตรวจสอบโครงสร้างข้อมูลและตรรกะเชิงวัตถุ

2. forms ตรวจสอบการตรวจสอบข้อมูลนำเข้า (Input Validation)

3. views ตรวจสอบการเรียกใช้ฟังก์ชัน การจัดการข้อมูล ความซับซ้อนของโค้ด

4. tests ประเมินความครอบคลุมของการทดสอบ (Code Coverage) จากชุดการทดสอบที่พัฒนาเพิ่มเติม

5. templates และ static ถูกละเว้นจากการวิเคราะห์ เพื่อมุ่งเน้นเฉพาะส่วนที่เป็นซอร์สโค้ดเชิงตรรกะของระบบ

การวิเคราะห์ด้วย SonarQube ใช้ตัวชี้วัดหลัก 4 ด้าน ได้แก่

1. Reliability การตรวจสอบข้อผิดพลาดเชิงตรรกะ (Bugs)

2. Maintainability ความซับซ้อนของโค้ดและจำนวน Code Smells

3. Coverage ความครอบคลุมของชุดการทดสอบ (ผ่าน pytest-cov integration)

4. Duplications การตรวจหาการเขียนโค้ดซ้ำซ้อน

4. ระยะที่ 3 (การเปรียบเทียบผลลัพธ์) ทำการเปรียบเทียบค่าตัวชี้วัดจาก SonarQube ระหว่างระบบก่อนปรับปรุง (T0) และระบบหลังปรับปรุง (T1) เพื่อวิเคราะห์ผลกระทบของการเพิ่ม Input Validation และ Automated Testing ต่อคุณภาพของซอร์สโค้ด ทั้งในด้านความถูกต้อง ความเสถียร ความสามารถในการบำรุงรักษา และความครอบคลุมของการทดสอบ

5. ตัวอย่างการวิเคราะห์ผลลัพธ์ (Example) จากการประเมินคุณภาพของระบบก่อนและหลังการปรับปรุงด้วย SonarQube พบว่า

Reliability ระบบหลังปรับปรุง (T1) มีจำนวนข้อผิดพลาดระดับ Bug ลดลงจาก 12 รายการ เหลือ 2 รายการ (ลดลงประมาณ 83%)

Maintainability ค่า Maintainability Rating ปรับจากระดับ C เป็นระดับ A โดยจำนวน Code Smells ลดลงกว่า 60%

Coverage ความครอบคลุมของการทดสอบเพิ่มจาก 0% ในระบบก่อนปรับปรุง (T0) เป็น 78.5% หลังการเพิ่มการทดสอบอัตโนมัติ (T1)

Duplications อัตราการซ้ำซ้อนของโค้ดลดลงจาก 5.8% เหลือ 1.2% เนื่องจากมีการปรับโครงสร้างฟังก์ชันและลดการเขียนซ้ำในหลายโมดูล

ผลการเปรียบเทียบดังกล่าวแสดงให้เห็นว่าแนวทางการปรับปรุงที่นำเสนอสามารถยกระดับคุณภาพของซอร์สโค้ดและเพิ่มความน่าเชื่อถือของระบบได้อย่างเป็นรูปธรรม

บทที่ 4
ผลการวิจัยและการอภิปรายผล

บทนี้นำเสนอผลการดำเนินงานตามขั้นตอนที่ได้อธิบายไว้ในบทที่ 3 ซึ่งประกอบด้วยการออกแบบ การพัฒนา และการปรับปรุงระบบต้นแบบ โดยเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Testing) เพื่อประเมินผลกระทบต่อคุณภาพของระบบสารสนเทศ ผลการดำเนินการจะถูกนำเสนอและวิเคราะห์ในเชิงเปรียบเทียบระหว่างระบบก่อนปรับปรุง (T0) และระบบหลังปรับปรุง (T1) โดยใช้เครื่องมือ SonarQube เป็นตัวประเมินเชิงปริมาณ ทั้งนี้ผลลัพธ์ที่ได้จะถูกนำมาใช้ในการวิเคราะห์เชิงคุณภาพและเชิงปริมาณ เพื่อสรุปประสิทธิภาพของแนวทางการปรับปรุงระบบในภาพรวม

4.1 ผลการพัฒนาระบบต้นแบบ

ระบบต้นแบบที่พัฒนาด้วย Django Framework มีวัตถุประสงค์เพื่อใช้เป็นเครื่องมือทดลองแนวทางการปรับปรุงคุณภาพของระบบสารสนเทศ โดยมีการพัฒนาในสองเวอร์ชันเพื่อใช้เปรียบเทียบผล ได้แก่

1. ระบบก่อนปรับปรุง (T0) ระบบที่ยังไม่มีการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Testing)

2. ระบบหลังปรับปรุง (T1) ระบบที่ได้เพิ่ม Input Validation และกระบวนการทดสอบแบบอัตโนมัติในหลายระดับ (Unit Test, Integration Test และ End-to-End Test)

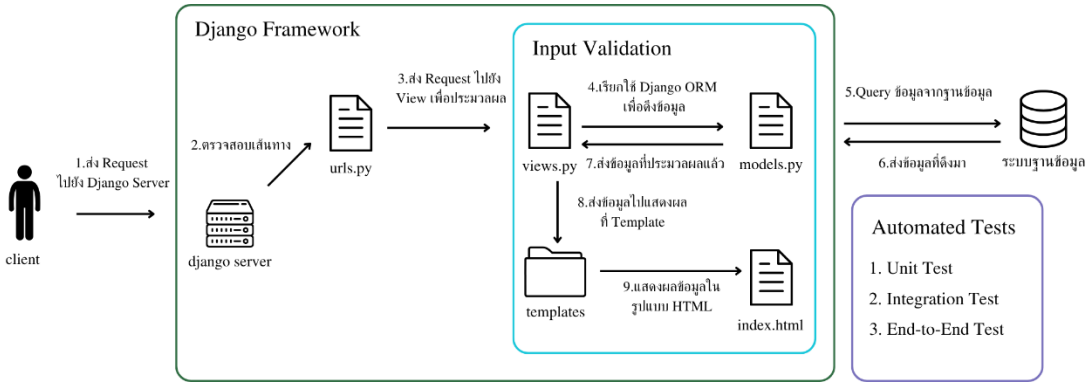
ระบบต้นแบบประกอบด้วย 3 โมดูลหลัก คือ

1. โมดูลการจัดการผู้ใช้ (User Management Module) ใช้สำหรับการลงทะเบียน การเข้าสู่ระบบ และการจัดการสิทธิ์ของผู้ใช้งาน

2. โมดูลการจัดการรายวิชา (Course Management Module) ใช้สำหรับเพิ่ม แก้ไข และลบข้อมูลรายวิชา รวมถึงจัดเก็บข้อมูลรายละเอียดของอาจารย์ผู้สอน

3. โมดูลการลงทะเบียนเรียน (Registration Module) ใช้สำหรับเชื่อมโยงข้อมูลระหว่างผู้ใช้กับรายวิชา และตรวจสอบความถูกต้องของข้อมูลก่อนการบันทึก

ระบบทั้งหมดถูกพัฒนาโดยใช้ ฐานข้อมูล PostgreSQL และสถาปัตยกรรมแบบ Model–View–Template (MVT) ของ Django เพื่อแยกส่วนการทำงานของระบบออกจากกันอย่างชัดเจน ช่วยให้สามารถพัฒนา และ ทดสอบได้อย่างเป็นระบบ



รูป 4.1 สถาปัตยกรรมของระบบ Django Framework ที่เพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Tests)

จากรูป 4.1 แสดงให้เห็นถึงสถาปัตยกรรมของระบบต้นแบบที่พัฒนาด้วย Django Framework ซึ่งใช้สถาปัตยกรรมแบบ Model–View–Template (MVT) เพื่อแยกส่วนการทำงานของระบบออกจากกันอย่างเป็นระบบและชัดเจน โดยลำดับการทำงานเริ่มจากผู้ใช้ (Client) ส่งคำร้องขอ (Request) ผ่าน Django Server เพื่อเข้าสู่กระบวนการตรวจสอบเส้นทาง (URL Mapping) จากไฟล์ urls.py และส่งคำขอไปยังส่วน View เพื่อประมวลผลคำร้อง ในขั้นตอนการประมวลผล View จะเรียกใช้งาน Model ผ่าน Django ORM เพื่อเข้าถึงและจัดการข้อมูลจากฐานข้อมูล PostgreSQL จากนั้นผลลัพธ์ที่ได้จะถูกส่งต่อไปยัง Template เพื่อแสดงผลในรูปแบบหน้าเว็บ HTML กลับไปยังผู้ใช้ ระบบนี้ยังรองรับการเพิ่มกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) ในระดับ View และ Model เพื่อควบคุมความถูกต้องของข้อมูลก่อนบันทึกลงฐานข้อมูล นอกจากนี้สถาปัตยกรรมดังกล่าวยังถูกออกแบบให้รองรับการเพิ่มกระบวนการ ทดสอบแบบอัตโนมัติ (Automated Dynamic Testing) ได้ในทุกระดับของระบบ โดยใช้เครื่องมือ Pytest สำหรับทดสอบในระดับโค้ด (Unit และ Integration) และ Robot Framework (Selenium) สำหรับการทดสอบเชิงพฤติกรรมของผู้ใช้จริง (End-to-End) ขณะที่เครื่องมือ SonarQube ถูกใช้ในการประเมินคุณภาพของซอร์สโค้ด เพื่อวัดค่าตัวชี้วัดความน่าเชื่อถือของระบบ (Reliability) ความสามารถในการบำรุงรักษา (Maintainability) ความครอบคลุมของการทดสอบ (Coverage) และอัตราการซ้ำซ้อนของโค้ด (Duplications) ดังนั้น สถาปัตยกรรมของระบบต้นแบบนี้ไม่เพียงแต่สะท้อนแนวทางการทำงานของ Django Framework เท่านั้น แต่ยังเป็นรากฐานสำคัญที่ช่วยให้สามารถดำเนินการทดสอบและประเมินคุณภาพของระบบสารสนเทศได้อย่างเป็นระบบและมีประสิทธิภาพมากยิ่งขึ้น

4.2 ผลการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation)

เพื่อเพิ่มความถูกต้องของข้อมูลและลดความผิดพลาดในกระบวนการทำงานของระบบต้นแบบ ได้มีการเพิ่มกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) ในส่วนของ View และ Model ของ Django Framework โดยใช้ฟังก์ชันในตัว เช่น clean() และคลาส validators เพื่อควบคุมรูปแบบและค่าของข้อมูลก่อนการบันทึกลงฐานข้อมูล PostgreSQL

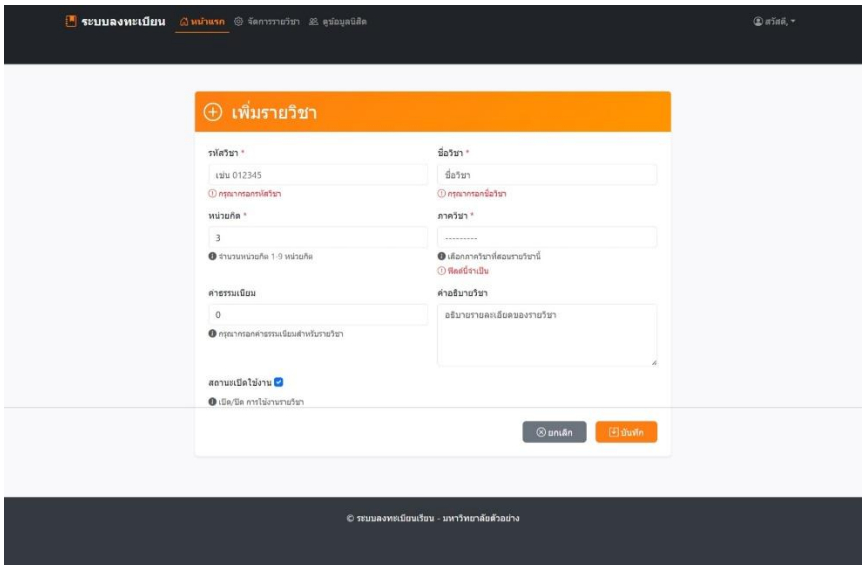
การตรวจสอบข้อมูลนำเข้าครอบคลุมกรณีทดสอบหลัก ดังนี้

1. การตรวจสอบรูปแบบข้อมูล (Format Validation) เช่น การตรวจสอบว่าอีเมลของผู้ใช้ต้องอยู่ในรูปแบบ name@example.com เท่านั้น หากไม่ตรงตามรูปแบบ ระบบจะแสดงข้อความแจ้งเตือนและไม่อนุญาตให้บันทึกข้อมูล

2. การตรวจสอบข้อมูลซ้ำ (Duplicate Validation) ระบบตรวจสอบว่ารายวิชาหรือรหัสผู้ใช้ที่กำลังเพิ่มมีอยู่แล้วในฐานข้อมูลหรือไม่ เพื่อป้องกันการบันทึกข้อมูลซ้ำซ้อน

3. การตรวจสอบค่าที่อยู่นอกขอบเขต (Range Validation) เช่น การจำกัดจำนวนหน่วยกิตของรายวิชาให้อยู่ในช่วงที่กำหนด หากผู้ใช้กรอกค่าที่เกินขอบเขต ระบบจะแสดงข้อความแจ้งเตือน

จากการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า พบว่าระบบสามารถป้องกันการบันทึกข้อมูลผิดพลาดเข้าสู่ฐานข้อมูลได้ทั้งหมด และช่วยลดการเกิดข้อผิดพลาดในกระบวนการทำงานของระบบในภายหลังได้อย่างมีนัยสำคัญ



รูป 4.2 ตัวอย่างผลการตรวจสอบข้อมูลนำเข้าในระบบต้นแบบ

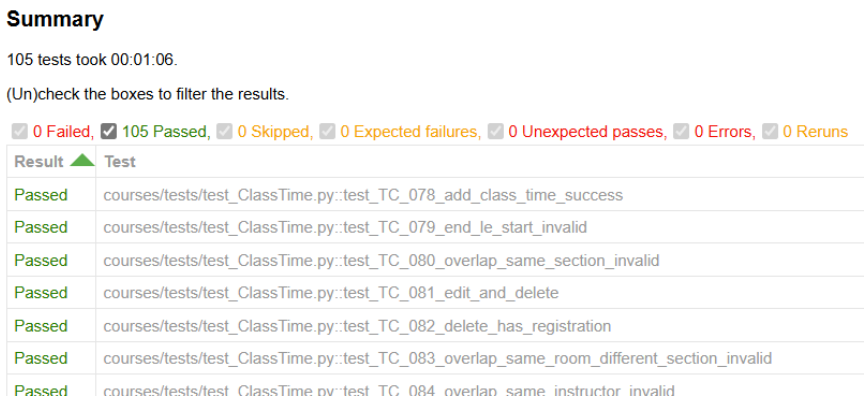
จากรูป 4.2 สามารถเห็นได้ว่าระบบต้นแบบสามารถตรวจจับข้อผิดพลาดของข้อมูลได้ตั้งแต่ขั้นตอนการกรอกข้อมูลของผู้ใช้ โดยระบบจะแสดงข้อความแจ้งเตือนแบบเรียลไทม์ (Real-time Validation) ผ่านหน้าเว็บ ช่วยให้ผู้ใช้สามารถแก้ไขข้อมูลให้ถูกต้องก่อนบันทึกจริง ผลลัพธ์ดังกล่าวช่วยลดข้อผิดพลาดของข้อมูลที่เข้าสู่ฐานข้อมูล และเพิ่มความน่าเชื่อถือของระบบในเชิงคุณภาพ

4.3 ผลการทดสอบแบบอัตโนมัติ (Automated Testing)

เพื่อประเมินความถูกต้องของการทำงานของระบบต้นแบบหลังการปรับปรุง ได้มีการเพิ่มกระบวนการ ทดสอบแบบอัตโนมัติ (Automated Testing) เพื่อจำลองการใช้งานจริงและตรวจสอบความถูกต้องของระบบในหลายระดับ โดยใช้เครื่องมือ Pytest และ Robot Framework (Selenium) ในการดำเนินการทดสอบ

4.3.1 การทดสอบระดับหน่วย (Unit Test) และการทดสอบแบบบูรณาการ (Integration Test)

การทดสอบระดับหน่วย (Unit Test) ดำเนินการโดยใช้ Pytest เพื่อทดสอบฟังก์ชันหลักของแต่ละโมดูล เช่น ฟังก์ชันการเพิ่มรายวิชา ฟังก์ชันการลงทะเบียนเรียน ฟังก์ชันการตรวจสอบรหัสผู้ใช้ ส่วนการทดสอบแบบบูรณาการ (Integration Test) ใช้สำหรับตรวจสอบการเชื่อมโยงระหว่างโมดูล เช่น ความสัมพันธ์ระหว่างผู้ใช้กับรายวิชา เพื่อให้แน่ใจว่าข้อมูลถูกส่งผ่านระหว่างโมดูลได้อย่างถูกต้องผลการทดสอบด้วย Pytest พบว่าฟังก์ชันทั้งหมดทำงานถูกต้องตามที่ออกแบบไว้ และผ่านการทดสอบในทุกกรณีที่กำหนด



Summary

105 tests took 00:01:06.

(Un)check the boxes to filter the results.

0 Failed, 105 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns

Result	Test
Passed	courses/tests/test_ClassTime.py::test_TC_078_add_class_time_success
Passed	courses/tests/test_ClassTime.py::test_TC_079_end_le_start_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_080_overlap_same_section_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_081_edit_and_delete
Passed	courses/tests/test_ClassTime.py::test_TC_082_delete_has_registration
Passed	courses/tests/test_ClassTime.py::test_TC_083_overlap_same_room_different_section_invalid
Passed	courses/tests/test_ClassTime.py::test_TC_084_overlap_same_instructor_invalid

รูป 4.3 ผลการทดสอบด้วย Pytest

จากรูป 4.3 สามารถเห็นได้ว่าการทดสอบด้วยเครื่องมือ Pytest ให้ผลลัพธ์เป็นบวกทั้งหมด โดยระบบผ่านการทดสอบทั้งในระดับหน่วย (Unit Test) และระดับบูรณาการ (Integration Test) ซึ่งครอบคลุมการทำงานของฟังก์ชันหลัก เช่น การเพิ่มรายวิชา การตรวจสอบข้อมูลการ

ลงทะเบียน และการเชื่อมโยงข้อมูลระหว่างโมดูลผู้ใช้กับรายวิชา ผลการทดสอบดังกล่าวยืนยันว่าระบบสามารถทำงานได้ถูกต้องตามที่ออกแบบไว้ในทุกส่วน โดยไม่มีข้อผิดพลาด (0 Failed) หรือข้อยกเว้น (0 Errors) ซึ่งแสดงให้เห็นถึงความถูกต้อง (Correctness) และความเสถียร (Stability) ของระบบหลังจากเพิ่มกระบวนการทดสอบแบบอัตโนมัติ

4.3.2 การทดสอบแบบ End-to-End (E2E Test)

เพื่อจำลองพฤติกรรมของผู้ใช้จริง ได้ใช้เครื่องมือ Robot Framework (Selenium) ในการทดสอบแบบ End-to-End (E2E) โดยระบบจะจำลองการทำงานตั้งแต่ขั้นตอนการเข้าสู่ระบบ (Login) การลงทะเบียนรายวิชา และการตรวจสอบข้อมูลในฐานข้อมูล ผลการทดสอบแสดงให้เห็นว่าระบบสามารถทำงานได้ครบถ้วนตามลำดับการใช้งานจริง โดยไม่มีข้อผิดพลาดในการแสดงผลหรือการเชื่อมโยงข้อมูล

สรุปผลการทดสอบแบบอัตโนมัติ จากผลการทดสอบทั้งสามระดับ พบว่าระบบหลังปรับปรุงมีความถูกต้องและความเสถียรในการทำงานสูงขึ้นอย่างชัดเจน โดยกระบวนการทดสอบแบบอัตโนมัติช่วยลดระยะเวลาในการตรวจสอบระบบ และสามารถทำการทดสอบซ้ำได้โดยไม่ต้องอาศัยการทดสอบด้วยตนเอง (Manual Testing) ซึ่งสอดคล้องกับแนวทางของ Thoughtworks (2012) ที่ระบุว่าการทดสอบอัตโนมัติช่วยเพิ่มประสิทธิภาพและลดข้อผิดพลาดจากมนุษย์ได้อย่างมีประสิทธิผล



Test Statistics

Total Statistics	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests	25	25	0	0	00:06:42	

Statistics by Tag	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
addsubject	4	4	0	0	00:01:10	
admin	1	1	0	0	00:00:13	
cancel	1	1	0	0	00:00:17	
delsubject	2	2	0	0	00:00:35	
detail	1	1	0	0	00:00:20	
duplicate	1	1	0	0	00:00:19	
editsubject	3	3	0	0	00:01:06	
enroll	1	1	0	0	00:00:13	
list	2	2	0	0	00:00:28	
login	7	7	0	0	00:01:29	
manage_courses	3	3	0	0	00:00:45	
navigation	1	1	0	0	00:00:13	
negative	12	12	0	0	00:03:16	
positive	6	6	0	0	00:01:39	
register	2	2	0	0	00:00:26	
search	2	2	0	0	00:00:36	
sections	1	1	0	0	00:00:21	
smoke	1	1	0	0	00:00:11	
student	1	1	0	0	00:00:12	
validation	8	8	0	0	00:02:08	
view_data	2	2	0	0	00:00:36	

Statistics by Suite	Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Tests	25	25	0	0	00:06:42	
Tests . 000 Test Login	7	7	0	0	00:01:29	
Tests . 001 Test Addsubject	4	4	0	0	00:01:10	
Tests . 002 Test Editsubject	3	3	0	0	00:01:06	
Tests . 003 Test Delsubject	2	2	0	0	00:00:35	
Tests . 004 Test Manage Courses	3	3	0	0	00:00:45	
Tests . 005 Test Viewdata	2	2	0	0	00:00:36	
Tests . 006 Test Register	2	2	0	0	00:00:26	
Tests . 007 Test Search	2	2	0	0	00:00:36	

รูป 4.4 ผลการทดสอบด้วย Robot Framework (Selenium)

4.4 ผลการประเมินคุณภาพของซอร์สโค้ดด้วย SonarQube

เพื่อประเมินคุณภาพของซอร์สโค้ดของระบบต้นแบบในเชิงปริมาณ ได้ใช้เครื่องมือ SonarQube ซึ่งเป็นเครื่องมือวิเคราะห์ซอร์สโค้ดแบบ Static Analysis สำหรับตรวจสอบคุณภาพ

ของซอฟต์แวร์ในด้านความถูกต้อง ความสามารถในการบำรุงรักษา และความครอบคลุมของการทดสอบ SonarQube จะวิเคราะห์ซอร์สโค้ดในสองช่วงเวลา คือ

1. ระบบก่อนปรับปรุง (T0)ระบบที่ยังไม่มี Input Validation และการทดสอบอัตโนมัติ

2. ระบบหลังปรับปรุง (T1) ระบบที่มีการเพิ่ม Input Validation และการทดสอบอัตโนมัติด้วย Pytest และ Robot Framework

4.4.1 ตัวชี้วัดที่ใช้ในการประเมินคุณภาพ (Metrics)

การประเมินคุณภาพของซอร์สโค้ดในงานวิจัยนี้อาศัยค่าตัวชี้วัดหลัก (Metrics) จากเครื่องมือ SonarQube ซึ่งใช้ในการวิเคราะห์คุณภาพซอฟต์แวร์เชิงสถิติ โดยพิจารณาจากสี่ปัจจัยหลัก ดังนี้

1. Reliability (ความน่าเชื่อถือของซอฟต์แวร์) หมายถึงระดับความถูกต้องและความเสถียรของระบบซอฟต์แวร์ โดย SonarQube จะให้คะแนนในรูปแบบของ Reliability Rating ตั้งแต่ระดับ A ถึง E ซึ่งสะท้อนถึงจำนวนและความรุนแรงของข้อบกพร่อง (Bugs) ที่ตรวจพบในซอร์สโค้ด

ระดับ A: ไม่มีข้อบกพร่องร้ายแรง หรือมีน้อยกว่า 0.1 Bugs ต่อ 1,000 บรรทัดโค้ด

ระดับ B: มีข้อบกพร่องน้อยกว่า 0.5 Bugs ต่อ 1,000 บรรทัดโค้ด

ระดับ C: มีข้อบกพร่องระดับปานกลาง

ระดับ D: มีข้อบกพร่องจำนวนมาก อาจส่งผลต่อการทำงานของระบบ

ระดับ E: มีข้อบกพร่องรุนแรงที่อาจทำให้ระบบล้มเหลว

ทั้งนี้ ยิ่งค่า Reliability Rating ใกล้ A มากเท่าใด หมายถึงระบบมีความถูกต้องและเสถียรสูง ลดความเสี่ยงจาก Runtime Errors และสะท้อนถึงคุณภาพของการออกแบบและการทดสอบที่ดี

2. Maintainability หมายถึงความง่ายในการปรับปรุง แก้ไข หรือขยายซอฟต์แวร์ โดยพิจารณาจากดัชนี Maintainability Rating (ระดับ A–E) และค่า Technical Debt Ratio (%) ซึ่งแสดงถึงสัดส่วนเวลาที่ต้องใช้ในการแก้ปัญหาเชิงโครงสร้างเมื่อเทียบกับเวลาพัฒนา

ระดับ A: Technical Debt < 5% (บำรุงรักษาง่ายมาก)

ระดับ B: Technical Debt 6–10% (บำรุงรักษาง่าย)

ระดับ C: Technical Debt 11–20% (พอใช้)

ระดับ D: Technical Debt 21–50% (ซับซ้อน ต้องปรับปรุง)

ระดับ E: Technical Debt > 50% (บำรุงรักษายากมาก)

ยิ่งค่า Maintainability Rating ต่ำ (ใกล้ A) หมายถึงซอร์สโค้ดมีความสะอาด ไม่ซ้ำซ้อน และสามารถดูแลต่อยอดได้ง่าย ซึ่งสะท้อนถึงคุณภาพการเขียนโค้ดที่ดี

3. Coverage หมายถึงร้อยละของบรรทัดโค้ดที่ถูกทดสอบโดยกระบวนการทดสอบอัตโนมัติ (Automated Tests) ค่าตัวชี้วัดนี้อยู่ในช่วง 0–100% โดย SonarQube คำนวณจากการรวมผลการทดสอบระดับหน่วย (Unit Test) และแบบบูรณาการ (Integration Test)

Coverage ≥ 80%: ครอบคลุมดีมาก ระบบมีการทดสอบอย่างทั่วถึง

Coverage 60–79%: ครอบคลุมในระดับดี

Coverage 40–59%: ครอบคลุมปานกลาง ควรเพิ่มกรณีทดสอบ

Coverage < 40%: การทดสอบไม่เพียงพอ มีความเสี่ยงสูง

ค่าที่สูงบ่งชี้ว่าระบบผ่านการตรวจสอบฟังก์ชันส่วนใหญ่ ทำให้ลดโอกาสเกิดข้อผิดพลาดในอนาคตและช่วยเพิ่มความมั่นใจในการปรับปรุงระบบ

4. Duplications หมายถึงร้อยละของบรรทัดโค้ดที่ถูกซ้ำซ้อนภายในระบบ (Duplicated Lines %) ค่านี้อยู่ในช่วง 0–100% โดยยิ่งค่าน้อยยิ่งดี

0–3%: ดีมาก โค้ดไม่ซ้ำซ้อน มีการออกแบบที่เป็นโมดูลและใช้ซ้ำได้

4–10%: อยู่ในระดับยอมรับได้ แต่ควรปรับปรุงให้โค้ดเรียบง่ายขึ้น

>10%: มีโค้ดซ้ำซ้อนมาก เสี่ยงต่อบัคและทำให้บำรุงรักษายาก

การลดค่า Duplications แสดงถึงการออกแบบเชิงโครงสร้างที่ดีและช่วยลดภาระในการแก้ไขโค้ดซ้ำหลายจุด

การพิจารณาค่าตัวชี้วัดทั้งสี่นี้ช่วยให้สามารถประเมินคุณภาพของระบบในหลายมิติ ทั้งด้านความถูกต้อง (Reliability) ความสามารถในการบำรุงรักษา (Maintainability) ความครอบคลุมของการทดสอบ (Coverage) และประสิทธิภาพของโครงสร้างโค้ด (Duplications) ซึ่งสะท้อนถึงประสิทธิภาพโดยรวมของระบบหลังการปรับปรุงได้อย่างเป็นรูปธรรม

4.4.2 ผลการประเมินคุณภาพของระบบก่อนและหลังการปรับปรุง

ผลการประเมินคุณภาพด้วยเครื่องมือ SonarQube แสดงใน ตาราง 4.1 และ รูป 4.5 แสดง Dashboard ของ SonarQube ที่ใช้ในการวิเคราะห์คุณภาพซอร์สโค้ดของระบบก่อนและหลังการปรับปรุง

ตาราง 4.1 ผลการประเมินคุณภาพของซอร์สโค้ดก่อนและหลังการปรับปรุงระบบด้วยเครื่องมือ SonarQube

ตัวชี้วัด (Metric)	ระบบก่อนปรับปรุง (T0)	ระบบหลังปรับปรุง (T1)	สรุปผลการเปรียบเทียบ
Reliability	3	0	ลดลง แสดงว่าระบบมีความถูกต้องและเสถียรมากขึ้น
Maintainability	37	33	ลดลง แสดงว่าซอร์สโค้ดมีคุณภาพและบำรุงรักษาง่ายขึ้น
Coverage	0.0%	78.3%	เพิ่มขึ้น แสดงว่าการทดสอบครอบคลุมระบบมากขึ้น
Duplications	8.0%	7.2%	ลดลง แสดงว่าโค้ดซ้ำซ้อนน้อยลง

จากตาราง 4.1 พบว่าระบบหลังการปรับปรุง (T1) มีคุณภาพของซอร์สโค้ดดีขึ้นในทุกตัวชี้วัด โดยค่า Reliability ลดลงจาก 3 เหลือ 0 ซึ่งหมายถึงระบบมีข้อผิดพลาดเชิงตรรกะน้อยลงและมีความเสถียรมากขึ้น ส่วนค่า Maintainability ลดลงจาก 37 เหลือ 33 แสดงให้เห็นว่าซอร์สโค้ดมีความเป็นระเบียบและสามารถบำรุงรักษาได้ง่ายกว่าเดิม สำหรับค่า Coverage เพิ่มขึ้นจาก 0.0% เป็น 78.3% แสดงให้เห็นว่าระบบได้รับการทดสอบครอบคลุมในหลายส่วนของโค้ดมากขึ้น ขณะที่ค่า Duplications ลดลงจาก 8.0% เหลือ 7.2% ซึ่งสะท้อนถึงการลดความซ้ำซ้อนของโค้ดและปรับปรุงคุณภาพเชิงโครงสร้างได้ดีขึ้น



รูป 4.5 ผลการประเมินคุณภาพของซอร์สโค้ดก่อนและหลังการปรับปรุงระบบด้วย SonarQube

ผลการประเมินดังกล่าวแสดงให้เห็นว่า การเพิ่มกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) และ การทดสอบแบบอัตโนมัติ (Automated Testing) ส่งผลให้ระบบสารสนเทศมีคุณภาพโดยรวมดีขึ้น ทั้งในด้านความถูกต้อง ความน่าเชื่อถือ และความสามารถในการบำรุงรักษา ซึ่งสอดคล้องกับหลักการของ SonarSource (2024) ที่ระบุว่าการผสานการทดสอบอัตโนมัติร่วมกับการวิเคราะห์ซอร์สโค้ดช่วยยกระดับคุณภาพของซอฟต์แวร์ได้อย่างมีประสิทธิภาพ

4.4.3 การวิเคราะห์ผลการประเมินคุณภาพ

จากผลการประเมินคุณภาพของซอร์สโค้ดด้วยเครื่องมือ SonarQube พบว่าระบบหลังการปรับปรุง (T1) มีคุณภาพโดยรวมดีกว่าระบบก่อนการปรับปรุง (T0) อย่างชัดเจน ทั้งในด้านความถูกต้อง ความสามารถในการบำรุงรักษา และความครอบคลุมของการทดสอบ ซึ่งสามารถวิเคราะห์ได้ดังนี้

1. ความน่าเชื่อถือของระบบ ค่า Reliability ลดลงจาก 3 เหลือ 0 แสดงให้เห็นว่าระบบหลังปรับปรุงมีข้อผิดพลาดเชิงตรรกะและข้อบกพร่องทางการทำงานน้อยลง ซึ่งเป็นผลโดยตรงจากการเพิ่มกระบวนการ ตรวจสอบข้อมูลนำเข้า (Input Validation) ที่ช่วยลดโอกาสการเกิดข้อผิดพลาดของข้อมูลภายในระบบ และการใช้ Pytest ในการทดสอบฟังก์ชันย่อย (Unit Test) ทำให้สามารถตรวจจับข้อผิดพลาดได้ตั้งแต่ขั้นตอนการพัฒนา

2. ความสามารถในการบำรุงรักษา ค่า Maintainability ลดลงจาก 37 เหลือ33 สะท้อนถึงโครงสร้างซอร์สโค้ดที่เป็นมาตรฐานมากขึ้น การใช้ Django Framework ที่มีรูปแบบโครงสร้างแบบ Model–View–Template (MVT) ช่วยให้สามารถแยกส่วนการทำงานออกจากกันได้อย่างชัดเจน ทำให้โค้ดอ่านง่ายและดูแลได้สะดวก นอกจากนี้ การเพิ่มการทดสอบแบบบูรณาการ (Integration Test) ยังช่วยให้ผู้พัฒนาสามารถระบุส่วนของระบบที่ต้องแก้ไขได้อย่างแม่นยำ ส่งผลให้การบำรุงรักษาในระยะยาวมีประสิทธิภาพมากขึ้น

3. ความครอบคลุมของการทดสอบ ค่า Coverage เพิ่มขึ้นจาก 0.0% เป็น 78.3% ซึ่งเป็นการเปลี่ยนแปลงที่มีนัยสำคัญ แสดงให้เห็นว่าการเพิ่มกระบวนการทดสอบแบบอัตโนมัติ โดยเฉพาะการใช้ Robot Framework (Selenium) สำหรับการทดสอบแบบ End-to-End ช่วยให้สามารถจำลองพฤติกรรมของผู้ใช้จริงและทดสอบระบบได้อย่างครอบคลุมในหลายกรณีการทำงาน ส่งผลให้คุณภาพโดยรวมของระบบเพิ่มสูงขึ้น

4. อัตราการซ้ำซ้อนของโค้ด ค่า Duplications ลดลงจาก 8.0% เหลือ 7.2% แสดงถึงการจัดการซอร์สโค้ดที่มีประสิทธิภาพมากขึ้นและลดความซ้ำซ้อนของโค้ดภายในระบบ ซึ่งเป็นผลมาจากการปรับปรุงโครงสร้างการเขียนโค้ดให้สอดคล้องกับหลักการของ Django Framework และแนวทางการบำรุงรักษาซอฟต์แวร์ที่ดี (Clean Code Practice)

สรุปได้ว่าการเพิ่มกระบวนการ Input Validation และ Automated Testing มีส่วนช่วยอย่างมากในการยกระดับคุณภาพของระบบสารสนเทศในหลายมิติ ทั้งด้านความถูกต้อง (Reliability) ความสามารถในการบำรุงรักษา (Maintainability) และความครอบคลุมของการทดสอบ (Coverage) ซึ่งสอดคล้องกับแนวทางของ SonarSource (2024) ที่ระบุว่าการผสานกระบวนการทดสอบอัตโนมัติร่วมกับการวิเคราะห์ซอร์สโค้ดช่วยเพิ่มคุณภาพและลดข้อผิดพลาดของซอฟต์แวร์ได้อย่างมีประสิทธิภาพ

4.5 การวิเคราะห์ผลการปรับปรุงระบบ

จากผลการประเมินคุณภาพของซอร์สโค้ดด้วยเครื่องมือ SonarQube พบว่าระบบหลังการปรับปรุง (T1) มีค่าความน่าเชื่อถือ (Reliability) และความสามารถในการบำรุงรักษา (Maintainability) สูงกว่าระบบก่อนการปรับปรุง (T0) อย่างชัดเจน ขณะที่ค่าความครอบคลุมของการทดสอบ (Coverage) เพิ่มขึ้น และอัตราการซ้ำซ้อนของโค้ด (Duplications) ลดลง แสดงให้เห็นว่าการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบอัตโนมัติ (Automated Testing) สามารถช่วยยกระดับคุณภาพของซอร์สโค้ดได้จริงในหลายมิติ

ในเชิงเทคนิค การปรับปรุงที่มีผลต่อ Reliability มาจากการเพิ่มการตรวจสอบข้อมูลนำเข้าด้วยฟังก์ชัน clean() และ validators ของ Django ซึ่งช่วยป้องกันข้อมูลผิดรูปแบบและลดข้อผิดพลาดที่อาจเกิดขึ้นจากผู้ใช้โดยตรง นอกจากนี้ การเขียน Test Cases ด้วย Pytest และ Robot Framework ยังช่วยตรวจสอบการทำงานของระบบในหลายระดับ ตั้งแต่หน่วยย่อย (Unit Test) การเชื่อมโยงระหว่างโมดูล (Integration Test) จนถึงการทำงานครบวงจร (End-to-End Test) ทำให้สามารถระบุและแก้ไขจุดบกพร่องได้ตั้งแต่ระยะต้นของการพัฒนา ส่งผลให้โค้ดที่ได้มีความเสถียรและลดโอกาสการเกิดบัคในระยะยาว

เมื่อพิจารณาตามแนวคิด Testing Pyramid การจัดสัดส่วนการทดสอบให้เหมาะสมในแต่ละระดับส่งผลให้การทดสอบมีความครอบคลุมและคุ้มค่า ทั้งในด้านเวลาและทรัพยากร โดยผลจากการทดสอบพบว่า Test Coverage มีแนวโน้มเพิ่มขึ้นอย่างต่อเนื่องหลังการเพิ่มชุดทดสอบในระดับ Unit และ Integration ซึ่งสอดคล้องกับแนวทางของ (Cohn, 2010) และ (Fowler, 2019) ที่ระบุว่าการเพิ่มสัดส่วนของ Unit Test จะช่วยลดต้นทุนการแก้ไขบัคและเพิ่มความน่าเชื่อถือของระบบโดยรวม

ในส่วนของ Maintainability ที่ดีขึ้น เกิดจากการจัดโครงสร้างโค้ดให้เป็นมาตรฐานตามรูปแบบ Model–View–Template (MVT) ของ Django ร่วมกับการลดโค้ดซ้ำซ้อนและการใช้ฟังก์ชันที่สามารถนำกลับมาใช้ซ้ำได้ (Reusable Functions) ซึ่งช่วยให้ระบบสามารถบำรุงรักษาและขยายเพิ่มเติมได้ง่ายในอนาคต ทั้งนี้ SonarQube ยังแสดงค่าการลดลงของ “Code Smells” ซึ่งเป็นตัวชี้วัดโดยตรงของคุณภาพเชิงโครงสร้าง

อย่างไรก็ตาม การทดลองครั้งนี้ยังมีข้อจำกัดบางประการ เช่น การประเมินคุณภาพของซอร์สโค้ดดำเนินการในสภาพแวดล้อมจำลอง (Local Environment) ที่ไม่มีปัจจัยแวดล้อมจากระบบจริง เช่น การเชื่อมต่อภายนอก หรือการโหลดข้อมูลขนาดใหญ่ อีกทั้งขอบเขตของการทดสอบยังเน้นเฉพาะการตรวจสอบความถูกต้องของข้อมูลและพฤติกรรมการทำงานของระบบ โดยไม่ได้รวมการทดสอบด้านประสิทธิภาพ (Performance Testing) หรือความปลอดภัย (Security Testing)

โดยสรุป การปรับปรุงระบบด้วยการเพิ่ม Input Validation และ Automated Testing ช่วยเพิ่มประสิทธิภาพและคุณภาพของระบบสารสนเทศได้อย่างมีนัยสำคัญ ทั้งในด้านความถูกต้อง เสถียรภาพ และความสามารถในการบำรุงรักษา ซึ่งเป็นแนวทางที่สามารถประยุกต์ใช้กับระบบสารสนเทศอื่น ๆ ได้ในอนาคต เพื่อยกระดับมาตรฐานการพัฒนาและควบคุมคุณภาพซอฟต์แวร์ในระดับองค์กร

บทที่ 5
สรุปผลการวิจัยและข้อเสนอแนะ

บทนี้เป็นการสรุปผลการดำเนินงานวิจัยทั้งหมดจากการทดลองที่ได้ดำเนินการในบทที่ 4 โดยมุ่งนำเสนอผลลัพธ์ที่ได้จากการปรับปรุงระบบต้นแบบด้วยการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบแบบอัตโนมัติ เพื่อประเมินผลการเปลี่ยนแปลงของคุณภาพซอร์สโค้ดและประสิทธิภาพของระบบ นอกจากนี้ยังได้อภิปรายถึงข้อจำกัดที่พบระหว่างการดำเนินงาน ตลอดจนเสนอแนะแนวทางการประยุกต์ใช้และการพัฒนาต่อยอดงานวิจัยในอนาคต เพื่อให้ระบบสารสนเทศมีคุณภาพและความเสถียรที่ดียิ่งขึ้น

5.1 สรุปผลการวิจัย

งานวิจัยนี้มีวัตถุประสงค์เพื่อปรับปรุงประสิทธิภาพและคุณภาพของระบบสารสนเทศ โดยการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และกระบวนการทดสอบแบบอัตโนมัติ (Automated Testing) เพื่อให้ระบบมีความถูกต้อง เสถียร และลดข้อผิดพลาดที่เกิดจากผู้ใช้และซอร์สโค้ดของโปรแกรม โดยได้ใช้ระบบลงทะเบียนเรียนที่พัฒนาด้วย Django Framework เป็นระบบต้นแบบสำหรับการทดลอง ทั้งนี้ได้ทำการสร้างระบบสองเวอร์ชัน ได้แก่

1. ระบบก่อนปรับปรุง (T0) ที่ยังไม่มีการตรวจสอบข้อมูลนำเข้าและไม่มีการทดสอบอัตโนมัติ
2. ระบบหลังปรับปรุง (T1) ที่ได้เพิ่ม Input Validation และเพิ่มการทดสอบอัตโนมัติในหลายระดับ ได้แก่ Unit Test, Integration Test และ End-to-End Test

ผลการทดลองและประเมินคุณภาพซอร์สโค้ดด้วยเครื่องมือ SonarQube พบว่า ระบบหลังการปรับปรุง (T1) มีคุณภาพของซอร์สโค้ดและประสิทธิภาพโดยรวมดีขึ้นอย่างชัดเจนเมื่อเทียบกับระบบก่อนปรับปรุง (T0) โดยมีผลการเปรียบเทียบดังนี้

1. Reliability ปรับปรุงจากระดับ C เป็นระดับ A โดยจำนวนข้อผิดพลาด (Bugs) ลดลงจาก 3 รายการเหลือ 0 รายการ แสดงให้เห็นว่าระบบมีความถูกต้องและเสถียรยิ่งขึ้นหลังการเพิ่ม Input Validation และ Test Script
2. Maintainability อยู่ในระดับ A ทั้งสองเวอร์ชัน แต่จำนวน Code Smells ลดลงจาก 37 รายการเหลือ 33 รายการ สะท้อนถึงการออกแบบโค้ดที่สะอาดและบำรุงรักษาง่ายขึ้น แม้ระบบจะมีขนาดใหญ่ขึ้นจาก 2.6k เป็น 2.9k บรรทัด
3. Coverage เพิ่มขึ้นอย่างมีนัยสำคัญจาก 0.0% เป็น 78.3% หลังการเพิ่มกระบวนการทดสอบแบบอัตโนมัติด้วย Pytest และ Robot Framework แสดงว่าการทดสอบครอบคลุมฟังก์ชันหลักของระบบเกือบทั้งหมด

4. Duplications ลดลงจาก 8.0% เหลือ 7.2% แสดงถึงการลดความซ้ำซ้อนของโค้ดและการปรับปรุงโครงสร้างให้มีความเป็นมาตรฐานมากขึ้น

5. Security พบความเสี่ยงเท่าเดิม 1 รายการในทั้งสองเวอร์ชัน ซึ่งสอดคล้องกับขอบเขตการวิจัยที่ไม่ได้เน้นการทดสอบด้านความปลอดภัย

จากผลการทดลองดังกล่าวสามารถสรุปได้ว่า การเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบอัตโนมัติช่วยปรับปรุงคุณภาพของระบบสารสนเทศได้อย่างมีประสิทธิภาพ โดยเฉพาะในด้านความถูกต้อง (Accuracy) ความน่าเชื่อถือ (Reliability) และความครอบคลุมของการทดสอบ (Coverage) ทั้งนี้ Django Framework มีบทบาทสำคัญในการสนับสนุนกลไกการตรวจสอบข้อมูลนำเข้า เช่น การใช้เมธอด clean() และ validators สำหรับควบคุมความถูกต้องของข้อมูลก่อนจัดเก็บในฐานข้อมูล ทำให้ลดความผิดพลาดจากการป้อนข้อมูลของผู้ใช้ได้อย่างมีนัยสำคัญ

ในภาพรวม การปรับปรุงระบบตามแนวทางที่เสนอส่งผลให้ซอร์สโค้ดมีคุณภาพดีขึ้นทั้งเชิงโครงสร้างและการทดสอบ ระบบสามารถทำงานได้อย่างถูกต้องมากขึ้น มีความครอบคลุมของการทดสอบที่ชัดเจน และพร้อมสำหรับการบำรุงรักษาในอนาคต ผลลัพธ์ที่ได้ยืนยันว่าการประยุกต์ใช้ Input Validation และ Automated Testing เป็นแนวทางที่มีประสิทธิผลในการยกระดับคุณภาพและประสิทธิภาพของระบบสารสนเทศในสภาพแวดล้อมจริง

5.2 ข้อจำกัดของงานวิจัย

1. ข้อจำกัดด้านสภาพแวดล้อมการทดลอง การทดลองทั้งหมดดำเนินการภายในสภาพแวดล้อมจำลอง (Local Environment) ซึ่งมีการควบคุมปัจจัยภายนอก เช่น เครือข่ายและปริมาณผู้ใช้งาน เพื่อให้การวัดผลมีความแม่นยำ แต่ขณะเดียวกันก็ทำให้ไม่สามารถสะท้อนประสิทธิภาพของระบบในสภาพแวดล้อมการใช้งานจริงได้อย่างสมบูรณ์ โดยเฉพาะในระบบขนาดใหญ่ที่มีผู้ใช้งานพร้อมกันหลายคน

2. ข้อจำกัดด้านขอบเขตของการทดสอบ งานวิจัยนี้มุ่งเน้นเฉพาะการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้า (Input Validation) และการทดสอบแบบพลวัตอัตโนมัติ (Automated Dynamic Testing) โดยไม่ได้ครอบคลุมการทดสอบในมิติอื่น เช่น การทดสอบสมรรถนะของระบบ (Performance Testing), การทดสอบความปลอดภัย (Security Testing) ทำให้ไม่สามารถประเมินคุณภาพของระบบในเชิงลึกทุกด้านได้ครบถ้วน

3. ข้อจำกัดของเครื่องมือวิเคราะห์คุณภาพซอร์สโค้ด เครื่องมือ SonarQube ที่ใช้ในการประเมินคุณภาพซอร์สโค้ดในงานวิจัยนี้ มุ่งเน้นการวิเคราะห์เชิงสถิติในด้าน Reliability, Maintainability, Coverage และ Duplications เท่านั้น โดยไม่ได้ครอบคลุมการประเมินด้านความปลอดภัยเชิงลึก เช่น การตรวจจับช่องโหว่ของระบบ (Vulnerability Detection) หรือการวิเคราะห์

Dependency ของแพ็กเกจภายนอก ซึ่งอาจทำให้การประเมินคุณภาพยังไม่ครอบคลุมในทุกมิติของซอฟต์แวร์

4. ข้อจำกัดด้านขนาดของระบบต้นแบบ ระบบต้นแบบที่พัฒนาในงานวิจัยนี้มีขนาดข้อมูลและจำนวนโมดูลจำกัด เพื่อให้เหมาะสมกับการทดลองในระดับวิชาการ ดังนั้นผลลัพธ์ที่ได้อาจแตกต่างจากระบบขนาดใหญ่ที่มีความซับซ้อนมากกว่า เช่น ระบบฐานข้อมูลแบบกระจาย หรือระบบที่มีการเชื่อมต่อกับบริการภายนอก (API Integration)

5.3 ข้อเสนอแนะงานวิจัย

จากผลการดำเนินงานและข้อจำกัดที่พบในงานวิจัยฉบับนี้ ผู้วิจัยมีข้อเสนอแนะสำหรับการพัฒนาและขยายผลในอนาคต ดังนี้

1. ขยายขอบเขตการทดสอบให้ครอบคลุมมิติอื่นของคุณภาพซอฟต์แวร์ งานวิจัยในอนาคตควรเพิ่มการทดสอบในมิติอื่นนอกเหนือจาก Input Validation และ Automated Dynamic Testing เช่น การทดสอบสมรรถนะของระบบ (Performance Testing) ด้วยเครื่องมืออย่าง Apache JMeter เพื่อประเมินความเร็วและความเสถียรเมื่อมีผู้ใช้งานพร้อมกันจำนวนมาก การทดสอบความปลอดภัย (Security Testing) โดยใช้เครื่องมือ OWASP ZAP หรือ Bandit เพื่อค้นหาช่องโหว่ในโค้ดและตรวจสอบการโจมตีจากภายนอก เช่น SQL Injection หรือ Cross-Site Scripting (XSS) การเพิ่มการทดสอบในมิติเหล่านี้จะช่วยให้การประเมินคุณภาพของระบบมีความครอบคลุมและสมบูรณ์มากยิ่งขึ้น

2. ประยุกต์แนวคิด Continuous Integration / Continuous Deployment (CI/CD) ควรนำเครื่องมือเช่น Jenkins, GitHub Actions หรือ GitLab CI มาใช้ผสานกับกระบวนการทดสอบอัตโนมัติ เพื่อให้ระบบสามารถตรวจสอบคุณภาพซอร์สโค้ดและรันการทดสอบได้ทุกครั้งที่มีการปรับปรุงโค้ด ซึ่งจะช่วยเพิ่มความต่อเนื่องและความน่าเชื่อถือของการพัฒนา (Continuous Quality Assurance)

3. พัฒนาระบบต้นแบบให้มีความซับซ้อนและใกล้เคียงกับระบบจริงมากขึ้น งานวิจัยต่อไปควรขยายขนาดของระบบต้นแบบให้มีโมดูลและข้อมูลจำนวนมากขึ้น เช่น การเชื่อมต่อกับระบบภายนอก (External API) หรือระบบจัดการผู้ใช้หลายระดับ (User Roles) เพื่อทดสอบประสิทธิภาพของแนวทางที่เสนอในบริบทของระบบขนาดใหญ่และซับซ้อนกว่าเดิม

4. เพิ่มการเปรียบเทียบเครื่องมือและเทคโนโลยีทางเลือก ควรทดลองเปรียบเทียบประสิทธิภาพของเครื่องมือวิเคราะห์คุณภาพซอร์สโค้ดและเครื่องมือทดสอบอัตโนมัติอื่น ๆ เช่น Semgrep, SonarLint, Cypress หรือ Playwright เพื่อประเมินว่าเครื่องมือใดให้ผลลัพธ์ที่แม่นยำและเหมาะสมที่สุดกับระบบที่พัฒนา

5. เพิ่มการประเมินในมุมมองของผู้ใช้งาน (User Acceptance Testing) ควรเก็บข้อมูลจากผู้ใช้จริงหรือผู้เชี่ยวชาญด้านซอฟต์แวร์ เพื่อประเมินความถูกต้อง ความเข้าใจง่าย และประสบการณ์การใช้งานของระบบ รวมถึงวิเคราะห์ผลกระทบของการเพิ่มกระบวนการตรวจสอบข้อมูลนำเข้าและการทดสอบอัตโนมัติต่อความพึงพอใจของผู้ใช้

6. พัฒนาแนวทางเป็น Framework หรือ Guideline สำหรับองค์กร ผลการวิจัยในครั้งนี้สามารถนำไปต่อยอดเป็นแนวทาง (Guideline) หรือเฟรมเวิร์กมาตรฐาน (Quality Assurance Framework) สำหรับองค์กรที่ต้องการปรับปรุงกระบวนการพัฒนาและทดสอบระบบสารสนเทศให้มีคุณภาพสูงขึ้น โดยเฉพาะในโครงการที่ใช้ Django Framework หรือเทคโนโลยี Python

บรรณานุกรม

Cohn, M. (2010). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley.

Django Software, F. (2024). Django documentation (version 5.x). สืบค้นเมื่อ 16 กันยายน 2568], จากเว็บไซต์: https://docs.djangoproject.com/en/stable/

Etemadi, K., Harrand, N., & Phu, H. L. (2021). Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations. *arXiv preprint arXiv:*2103.12033. https://arxiv.org/abs/2103.12033

Faisal, M., & Elshoush, H. (2023). Input Validation Vulnerabilities in Web Applications: Systematic Review, Classification and Analysis of the Current State-of-the-Art. *IEEE Access*. https://doi.org/10.1109/ACCESS.2023.3266385

Foundation, I. (2023). Certified tester foundation level syllabus (v4.0). สืบค้นเมื่อ 16 กันยายน 2568], จากเว็บไซต์: https://www.istqb.org/

Foundation, O. (2023). Input validation cheat sheet. สืบค้นเมื่อ 16 กันยายน 2568], จากเว็บไซต์: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Fowler, M. (2019). The Practical Test Pyramid. https://martinfowler.com/articles/practical-test-pyramid.html

Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Krekel, H. (2024). Pytest documentation (version 8.x). สืบค้นเมื่อ 16 กันยายน 2568], จากเว็บไซต์: https://docs.pytest.org/en/stable/

Krekel, H., Oliveira, B., Pfannschmidt, R., Bruynooghe, F., & pytest-dev, t. (2023). pytest Documentation (Version 7.4). https://docs.pytest.org/

Robot Framework, F. (2024). Robot Framework user guide (version 7.0). สืบค้นเมื่อ 10 ตุลาคม 2568], จากเว็บไซต์: https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html

SonarSource. (2024). SonarQube documentation (version 10.x). สืบค้นเมื่อ 10 ตุลาคม 2568], จากเว็บไซต์: https://docs.sonarsource.com/sonarqube/latest/

Thoughtworks. (2012). The testing pyramid. สืบค้นเมื่อ 10 ตุลาคม 2568], จากเว็บไซต์: https://martinfowler.com/articles/practical-test-pyramid.html

Usage of Robot Framework in Automation of Functional Test. (2014). สืบค้นเมื่อ 10 ตุลาคม 2568], จากเว็บไซต์: https://www.researchgate.net/publication/268185107

ภาคผนวก

ภาคผนวก ก ตัวอย่างโค้ดการตรวจสอบข้อมูลนำเข้า (Input Validation)

```python
def clean(self):
        cleaned_data = super().clean()
        section_number = cleaned_data.get('section_number')
        semester = cleaned_data.get('semester')
        room = cleaned_data.get('room')

        if section_number and semester and self.course:
            # สร้าง query เพื่อตรวจสอบการซ้ำซ้อน
            qs = Section.objects.filter(
                course=self.course,
                semester=semester,
                section_number=section_number
            )

            # ถ้าเป็นการแก้ไข ไม่นับตัวเองในการตรวจสอบ
            if self.instance.pk:
                qs = qs.exclude(pk=self.instance.pk)

            # ถ้ามีการซ้ำ ให้เพิ่ม error
            if qs.exists():
                self.add_error(
                    'section_number',
                    'กลุ่มเรียน (Sec) หมายเลขนี้มีอยู่แล้วในรายวิชาและภาคเรียนเดียวกัน'
                )
        return cleaned_data
```

รูป ก.1 ตัวอย่างฟังก์ชัน clean() สำหรับตรวจสอบการซ้ำของกลุ่มเรียน (Section)

ฟังก์ชัน clean() ใช้ใน Django Form เพื่อสร้างกระบวนการตรวจสอบข้อมูลก่อนบันทึกลงฐานข้อมูล โดยในตัวอย่างนี้ระบบจะตรวจสอบว่าหมายเลขกลุ่มเรียน (Section Number) ซ้ำภายในรายวิชาและภาคการเรียนเดียวกันหรือไม่ หากซ้ำจะขึ้นข้อความแจ้งเตือนเพื่อป้องกันการบันทึกข้อมูลซ้ำ

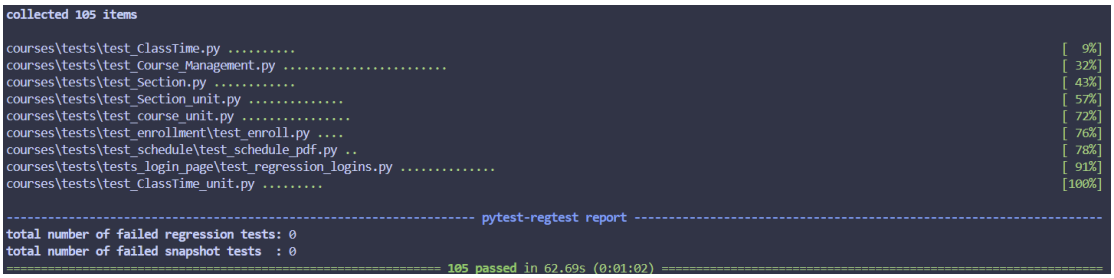


```python
code = models.CharField(
        max_length=10,
        unique=True,
        verbose_name="รหัสวิชา",
        validators=[
            RegexValidator(
                regex=r'^\d{6}$',
                message='รหัสวิชาต้องเป็นตัวเลข 6 หลัก'
            )
        ]
    )
```

รูป ก.2 ตัวอย่างการใช้ RegexValidator สำหรับตรวจสอบรูปแบบรหัสวิชา

คลาส RegexValidator ใน Django ใช้กำหนดรูปแบบของข้อมูลที่ต้องการตรวจสอบ โดยตัวอย่างนี้กำหนดให้รหัสวิชาต้องเป็นตัวเลขจำนวน 6 หลักเท่านั้น หากผู้ใช้กรอกรหัสไม่ตรงตามรูปแบบ ระบบจะแจ้งข้อความเตือน “รหัสวิชาต้องเป็นตัวเลข 6 หลัก”

ภาคผนวก ข ตัวอย่างผลการทดสอบอัตโนมัติด้วย Pytest และ Robot Framework



```
collected 105 items

courses\tests\test_ClassTime.py .........                                [  9%]
courses\tests\test_Course_Management.py ..........                       [ 32%]
courses\tests\test_Section.py ...........                                [ 43%]
courses\tests\test_Section_unit.py ..............                        [ 57%]
courses\tests\test_course_unit.py ................                       [ 72%]
courses\tests\test_enrollment\test_enroll.py ....                        [ 76%]
courses\tests\test_schedule\test_schedule_pdf.py ..                      [ 78%]
courses\tests\tests_login_page\test_regression_logins.py .............  [ 91%]
courses\tests\test_ClassTime_unit.py .........                           [100%]

---------- pytest-regtest report ----------
total number of failed regression tests: 0
total number of failed snapshot tests  : 0
========== 105 passed in 62.69s (0:01:02) ==========
```

รูป ข.1 ตัวอย่างผลการทดสอบด้วยเครื่องมือ Pytest

ภาพแสดงผลการรันคำสั่ง pytest สำหรับทดสอบระบบลงทะเบียนเรียนต้นแบบ โดยมีการรันทดสอบทั้งหมด 105 กรณี (Test Items) ครอบคลุมการทำงานของโมดูลหลัก ได้แก่ การจัดการ

รายวิชา (Course Management), กลุ่มเรียน (Section), การลงทะเบียน (Enrollment) และการสร้างเอกสารตารางเรียน (Schedule)

ผลการทดสอบทั้งหมด ผ่าน 100% (105 passed / 0 failed) ภายในเวลา 62.69 วินาที แสดงให้เห็นว่าฟังก์ชันของระบบทำงานถูกต้องและผ่านการตรวจสอบในทุกระดับของ Unit Test และ Integration Test ซึ่งสะท้อนถึงความถูกต้องและความเสถียรของระบบหลังการปรับปรุง



รูป ข.2 ตัวอย่างผลการทดสอบแบบ End-to-End ด้วย Robot Framework

ภาพแสดงผลการรันสคริปต์ทดสอบด้วย Robot Framework (Selenium) เพื่อจำลองพฤติกรรมการใช้งานจริงของผู้ใช้ เช่น ผู้ดูแลระบบ (Admin) สามารถเปิดหน้าจัดการรายวิชาและข้อมูลนักศึกษาได้ และนักศึกษาสามารถลงทะเบียนเรียน ค้นหารายวิชา และดูข้อมูลผลลัพธ์ได้ตามขั้นตอน ผลการทดสอบทั้งหมดผ่าน 100% (25 tests, 25 passed, 0 failed) แสดงถึงความถูกต้องและเสถียรของระบบในมุมมองของผู้ใช้งานจริง

ประวัติผู้วิจัย

ชื่อ – สกุล ปิติภัทร กิจสวน
วัน-เดือน-ปีเกิด 16 กันยายน 2546
ที่อยู่ 304/9 ซอย พิชัยสงคราม 12 ถนน พิชัยสงคราม
ตำบลในเมือง อำเภอเมือง จังหวัดพิษณุโลก 65000

ประวัติการศึกษา

วิทยาศาสตรบัณฑิต	สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยนเรศวร
มัธยมศึกษาตอนปลาย	โรงเรียนพุทธชินราชพิทยา ตำบลในเมือง อำเภอเมือง จังหวัดพิษณุโลก
มัธยมศึกษาตอนต้น	โรงเรียนพุทธชินราชพิทยา ตำบลในเมือง อำเภอเมือง จังหวัดพิษณุโลก

E-mail Address. blueblue160946@gmail.com